Breaking the Space Barrier and its Application to Language Model Inference

Arip Asadulaev[1] ✉

Date
October 2026

Code
https://github.com/Generated-Computers/Locals

Directed reachability is the problem behind nondeterministic logarithmic space, and Savitch's $O(\log^2 n)$ has been its deterministic space barrier for half a century. We study the total few-path version $\textsc{stcon}_{\mathrm{sf}}$, in which the algorithm receives an arbitrary graph and must itself verify that no two vertices are joined by more than k walks. We prove that $\textsc{stcon}_{\mathrm{sf}}$ lies in ReachUL, LOGDCFL, $\mathsf{C}_{=}\mathsf{L}$ and SC^2 and is decided in deterministic space $O(\log^2 M/\log\log M)$, below Savitch's bound, which answers a question of Apers and Edenhofer. All proofs rest on one lemma: walks no longer than the total budget decide the answer. We then show that these constructions are the control side of language-model inference. The machinery that enforces a grammar on a model's output has a step graph in which exactly one path joins any two vertices, and each proof becomes a decoding mechanism. Built into an engine for Apple's MLX and measured on one 16 GB laptop with Qwen3.5-2B and 4B, they give a schema-constrained stream that decodes faster than a free one (180 against 138 tokens/s) and as fast as XGrammar's jump-forward while keeping far more of the model's answer (0.87–0.94 of its values against 0.65–0.71), a 1,106-state tool grammar in 3 MB instead of 824 MB, and sixteen tool-calling agents that finish $2.55\times$ sooner.

1 Introduction

Given a directed graph and two of its vertices, is there a path from one to the other? This is the complete problem of nondeterministic logarithmic space (NL), and the best deterministic algorithm known for it, Savitch's, needs $O(\log^2 n)$ space. Whether that bound can be lowered is one of the oldest open questions in the field. The problem becomes easier when few paths join any two vertices, but every classical algorithm for that case has been given the few-path property as a promise. Apers & Edenhofer (2025) recently showed that a quantum computer with logarithmic memory needs no promise, and asked where the language sits classically.

The theorems. We answer the question for the *total* language, in which the input is an arbitrary graph and the algorithm must reject, within the same small memory, every graph that violates the few-path bound. It lies in four classes at once, ReachUL, LOGDCFL, $\mathsf{C}_{=}\mathsf{L}$ and SC^2, and is decided in deterministic space $O(\log^2 M/\log\log M)$, below Savitch's bound (Theorem 2). The proofs share one tool, a lemma that lets a search be cut at its total budget and still be complete or have found a violation (Lemma 3), and three constructions built on it. We also prove two facts about the shape of few-path graphs: between two vertices almost every vertex is on a forced chain, and the paths factor into independent blocks (Theorems 6 and 7).

The application. A language model writes text one token at a time, and on a laptop each token costs one reading of the whole model from memory, a *weight stream*. Increasingly the output must follow a grammar (a JSON schema, a typed tool call, a recursive document), and

[1]Generated Computers ✉e-mail: arip.asadulaev@mbzuai.ac.ae

Table 1 | Headline results. One Apple M2 Pro (16 GB), Qwen3.5-2B and 4B in 4-bit precision, greedy decoding. Each row names the result it follows from. Speed baselines run in our decode loop with the same kernels, except the last row, which compares whole systems.

Setting	Result used	STCON (ours)	Baseline
JSON schema, one stream, 2B / 4B	forced chains (Thm. 6)	**180 / 82** tok/s	free decoding: 138 / 63 tok/s
JSON schema, 2B / 4B	canonical split (Thm. 8)	**173 / 85** tok/s, fidelity **0.94 / 0.87**	XGrammar jump-forward: 159 / 86 tok/s, fidelity 0.65 / 0.71
Recursive schema, one stream, 2B	capped stack (Thm. 4)	**170** tok/s	llguidance: 94 tok/s
1,106-state typed tool grammar	walk kernel (ReachUL)	**3.4 MB**	token table: 824 MB, 62–127 s to build
16 grammars in one server, 4B	walk kernel (ReachUL)	**all 16**, +1.1 GB	token tables die at the 15th, +6.9 GB
JSON under a token limit, 2B	budget lemma (Lem. 3)	**90 / 90** valid	no budget rule: 3 / 90
18 outputs, weight streams, 2B / 4B	block product (Thm. 7)	**374 / 484**, identical	sequential: 672 / 686
16 tool-calling agents, 2B	whole engine	**29.3 s**	mlx-lm + llguidance: 74.6 s

several such agents run at once. The machinery that enforces the grammar is deterministic, and its graph of configurations has the few-path property in the strongest form: exactly one path joins any two vertices (Theorem 8). Every proof then becomes a mechanism. The capped traversal computes a grammar's whole token table in one pass; forced chains are output the engine writes without the model, which is why a constrained stream decodes faster than a free one, the recompute-rather-than-store discipline of the ReachUL proof removes the table altogether, the capped stack machine is the constraint for recursive schemas, and the block product lets independent parts of an output be decoded in parallel and certified exact. XGrammar (Dong et al., 2025) made grammar masks cheap on servers, and llguidance (Moskal et al., 2024) computes them lazily on the CPU. On a laptop two costs remain: a host-side mask forces a CPU-to-GPU round trip at every step, so mask-only decoding with either library is 25% slower than free decoding, and XGrammar's jump-forward re-tokenizes forced text, which is fast but keeps only 0.65–0.71 of the unconstrained answer's values. The one-path property removes both. Table 1 summarizes the results; each follows from a theorem.

Contributions and organization. (1) The four class memberships and the space bound for total few-path reachability, with the budget lemma that unifies the proofs (Section 2). (2) The structure theorems: branch kernel and block product. (3) The step-graph theorem, which identifies the control side of decoding as a mangrove (Section 3). (4) Six mechanisms derived from the proofs, built into an engine and measured for speed, quality and memory on one laptop (Sections 3 and 4). The main text is self-contained; Supplementary Section A explains the inference terms in plain words, Supplementary Sections B and C hold the proofs, Supplementary Section D the mechanisms in detail, and Supplementary Sections F and H every measurement.

2 Theory

2.1 Walks, few paths and the language

Definition 1 (Walks and few paths). Let G be a finite directed graph with n vertices. A *walk* from u to v is a sequence of edges that starts at u and ends at v; vertices may repeat, and the empty walk counts as one walk from every vertex to itself. $N_G(u, v) \in \mathbb{N} \cup \{\infty\}$ is the number of walks from u to v, and $N_G^{\leq L}(u, v)$ the number of those of length at most L. For $k \geq 1$, G is *k-strongly few* if $N_G(u, v) \leq k$ for every ordered pair, a 1-strongly-few graph is a *mangrove* (Allender & Lange, 1998).

$N_G(u, v)$ is infinite exactly when a cycle can be reached from u and can reach v; since the empty walk counts, a k-strongly-few graph has no cycles. The language is

$$\textsc{stcon}_{\text{sf}} = \left\{ \langle G, s, t, 1^k \rangle : \ N_G(u, v) \leq k \text{ for all } u, v, \text{ and } N_G(s, t) \geq 1 \right\},$$

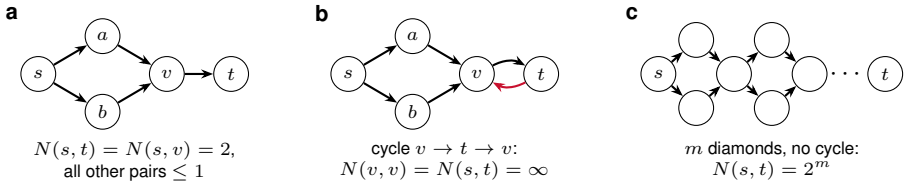


Fig. 1 | One good graph and the only two ways to be bad. **a**, $G_\star$ is 2-strongly few and t is reachable from s. **b**, $G_\circ$ adds one edge and with it a cycle, hence infinitely many walks. **c**, $G_\diamond$ has no cycle but 2^m walks. A total algorithm must reject $G_\circ$ and $G_\diamond$ with the memory, time and unambiguity it uses on $G_\star$.

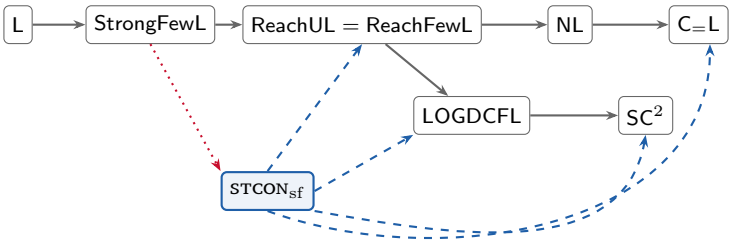


Fig. 2 | Where the language lands. Solid arrows are known inclusions, dashed blue arrows the memberships proved here, and the dotted red arrow the hardness; the upper row bounds the ambiguity of a nondeterministic logspace machine, the lower row gives a deterministic machine a stack. Whether NL is contained in LOGDCFL or in SC^2 is open, which is why the memberships need separate constructions. Membership in StrongFewL is open (Theorem C.5).

with k in unary and G given by its adjacency matrix; M is the input length, so $n \leq M$ and $nk \leq M^2$. The word *and* is the whole difficulty: the input may be any graph, and the algorithm must notice cycles and exponentially many walks within the memory it uses for good graphs. Fig. 1 shows one good graph and the only two ways to be bad.

2.2 The main theorem

Theorem 2 (Main theorem). *For unary k, $\textsc{stcon}_{\mathrm{sf}} \in \mathsf{ReachUL} \cap \mathsf{LOGDCFL} \cap \mathsf{C_{=}L} \cap \mathsf{SC}^2$, and $\textsc{stcon}_{\mathrm{sf}} \in \mathsf{DSPACE}\big(O(\log^2 M / \log\log M)\big)$.*

Four kinds of small-memory machine each decide whether a graph has few paths and whether t is reachable (Fig. 2): an unambiguous nondeterministic machine, for which no configuration is reachable by two computations (ReachUL; Garvin et al., 2014), a deterministic machine with a stack, in polynomial time (LOGDCFL; Sudborough, 1978), a machine that tests whether a signed count is zero ($\mathsf{C_{=}L}$), and one deterministic machine that is fast and small at the same time (SC^2). The space bound is the barrier result: general reachability is not known to be decidable in less than Savitch's $\log^2$ space, and here it is, by way of the unambiguous machine and the simulation of Allender & Lange (1998). The theorem does not put the problem in L and collapses no classes. Because the language is total, every algorithm for it *verifies* the few-path property, and that verification is what makes the constructions safe to reuse.

2.3 How the proofs go

Lemma 3 (Budget lemma). *Let H be a finite directed graph with a root r, give every vertex v a budget $\kappa(v) \in \mathbb{N}$, and let $B = \sum_v \kappa(v)$. Then $N_H(r,v) \leq \kappa(v)$ for all v if and only if $N_H^{\leq B}(r,v) \leq \kappa(v)$ for all v.*

Proof. Left to right is immediate. Conversely, suppose the truncated counts fit the budgets but some walk from r has length at least B. Its prefixes of lengths $0, \ldots, B$ are $B+1$ distinct walks of length at most B; since the budgets sum to B, some vertex v ends more than $\kappa(v)$ of them, a contradiction. So every walk from r is shorter than B, and $N_H = N_H^{\leq B}$. □

Explore walks only up to the total budget: either you have seen every walk, or you have already found a violation. There is no third case, and we say such a search *fails closed*. The bound cannot be lowered: on a cycle of B vertices with unit budgets the first violation appears exactly at depth B. The lemma speaks about one root, and a *universal source with copies* turns all pairs into one: a new root ρ with an edge to a copy (a,a) of every vertex, and a copy of

Table 2 | Each tool breaks on a bad graph; one safeguard per tool repairs it. One row per construction.

Tool	How a bad graph breaks it	Safeguard	Result
Depth-first search with a stack	circles forever on $G_\circ$; visits 2^m walks on $G_\diamond$	stop after $B+1$ nodes; count only after every tree is known to be small	LOGDCFL, SC^2
Hashing walks	too many walks force collisions, so counting by hash is wrong	certify that no hashes collide, by an unambiguous check; only then count	ReachUL, sub-Savitch space
Signed counting	counts may be infinite; "$\leq k$ for all pairs" is an AND	cut walks at length B, add squares of vanishing products	$\mathsf{C}_=\mathsf{L}$

G per tag a. Then $N(\rho,(a,v)) = N_G(a,v)$ (Lemma B.1), so $\textsc{stcon}_{\mathrm{sf}}$ is one rooted instance with total budget $1+n^2k$. Each classical tool breaks on a bad graph in its own way, and each construction adds one safeguard (Table 2).

Construction 1: a capped stack (LOGDCFL, SC^2). Walk the tree of all walks from a source depth first, keeping only a stack of return points (a vertex and the index of its next successor), and abort as soon as more than nk nodes have been entered.

Theorem 4 (Capped stack traversal). *The decider that runs this traversal from every source and rejects on overflow, then recounts the walks to every target and rejects when a count exceeds k, and accepts iff the (s,t) count is positive, accepts exactly $\textsc{stcon}_{\mathrm{sf}}$. It is a deterministic auxiliary pushdown automaton with an $O(\log M)$ work tape running in polynomial time, so $\textsc{stcon}_{\mathrm{sf}} \in \mathsf{LOGDCFL} \subseteq \mathsf{SC}^2$.*

Proof sketch. In a k-strongly-few graph the tree of walks from any source has at most nk nodes, so every traversal completes, the graph is acyclic, so the counts are exact. A source on a cycle has a walk of every length and overflows. Recounts start only after every source has bounded its tree, so a bad input cannot force an exponentially long count. Polynomial-time deterministic auxiliary pushdowns accept exactly LOGDCFL (Sudborough, 1978), and Cook (1979) places LOGDCFL in SC^2 (Supplementary Section B.3). □

Construction 2: certify, then count (ReachUL). A walk is named by a polynomial in the numbers of its edges, evaluated at a point α of a prime field, and the walks are lifted into a layered graph of triples (endpoint, length, hash). If no two walks collide, every lifted state has one root path and counting walks is counting reachable states. The machine first certifies that no collision happened, by a check that never becomes ambiguous, and only then counts.

Theorem 5 (Certified lift). *This controller decides $\textsc{stcon}_{\mathrm{sf}}$ by a reach-unambiguous logspace computation, so $\textsc{stcon}_{\mathrm{sf}} \in$ ReachUL. With the simulation of Allender & Lange (1998), the language is in $\mathsf{DSPACE}\big(O(\log^2 M/\log\log M)\big)$.*

Proof sketch. On a good input some α separates all walks, and every count is exact and within budget. On a bad input either no lift is certified, or a certified lift gives exact counts and Lemma 3 exposes the violation. Reach-unambiguity of a rooted graph is itself decided in ReachUL (Lange, 1997), and $\mathsf{L}^{\mathsf{ReachUL}} = \mathsf{ReachUL}$ (Garvin et al., 2014); Supplementary Section B.5 replaces the oracle by an explicit Euler tour that guesses only parents inside certified layers. □

Construction 3: one equation ($\mathsf{C}_=\mathsf{L}$). The truncated counts $f_{uv} = N^{\leq nk}(u,v)$ are #L functions. The product $\prod_{j=0}^{k}(f_{uv}-j)$ vanishes exactly when a count is within budget, and the sum of the squares of these products over all pairs, with $j \geq 1$ for the pair (s,t), is a GapL function that is zero exactly on $\textsc{stcon}_{\mathrm{sf}}$ (Theorem B.15).

2.4 The shape of few-path graphs

Restrict a graph to the vertices and edges on some s–t path, the result is a DAG H with $r = N_H(s,t)$ paths and cyclomatic number $c(H) = |E| - |V| + 1$.

Theorem 6 (Branch kernel). $c(H)+1 \leq r \leq 2^{c(H)}$. *Contracting every internal vertex with one way in and one way out leaves a multigraph with the same r paths, at most $2c+2 \leq 2r$ vertices and $3c+1$ edges. Between any two vertices of a k-strongly-few graph, almost everything is a forced chain: at most $2k$ vertices remain.*

Theorem 7 (Block product). *Let $B_1, \ldots, B_q$ be the biconnected blocks on the route from s to t, with entry and exit cut vertices a_i, b_i. Then $N_H(s,t) = \prod_i N_{B_i}(a_i, b_i)$ and a path is an independent choice of one a_i–b_i path per block.*

Proofs are in Supplementary Section C.4. Both facts are read directly off the graph of a decoding step below: forced chains are free output, and blocks are parts of an output that can be produced in parallel.

Takeaway: what the theorems hand to the engine

(1) A search stopped at its budget is complete or has found an error (Lemma 3), so every cap can be trusted. (2) In a one-path graph nothing needs to be visited twice or stored (Construction 2), so a table can be built in one sweep or not built at all. (3) A capped stack recognizes the language (Theorem 4), so recursion is a stack. (4) Between two vertices almost everything is forced and the rest splits into independent blocks (Theorems 6 and 7), so forced text is free and independent parts run in parallel.

3 Application: the control side of inference is a mangrove

3.1 What a language model does at every step

To choose the next token a language model runs its whole network once: the last token goes in, a score (a logit) for each of the 248,320 tokens of the vocabulary comes out, and the best token is taken or one is drawn from the scores. On a laptop this step is slow for one reason: all weights must be read from memory, 1.1 GB for Qwen3.5-2B and 2.26 GB for the 4B even at 4 bits, which takes 7 or 15 ms, and the arithmetic is nearly free next to it. Two things follow. A token written without running the model saves a whole weight stream, and a step that handles several tokens costs barely more than a step for one ($1.58\times$ for eight tokens on the 2B).

A constraint is compiled into a deterministic finite automaton (DFA) over bytes with a few hundred states. At every step the engine forbids the tokens whose bytes the DFA would reject (the *mask*) and, after the choice, moves the DFA along the bytes of the chosen token. Outlines keeps a table of states $\times$ vocabulary per grammar (Willard & Louf, 2023); XGrammar precomputes the tokens whose validity does not depend on the stack and checks the rest on the CPU (Dong et al., 2025); llguidance computes masks lazily on the CPU (Moskal et al., 2024).

3.2 Why the constraint has one path

Take any DFA state and any token. The token is a fixed string of bytes and the DFA is deterministic, so following those bytes leads to exactly one next state, or to a dead end. Store the vocabulary as a trie (0.59 million nodes for Qwen3.5), and let the *step graph* at configuration c pair each trie node w with the state the machine reaches on it, $P_c = \{(w, \delta^*(c,w))\}$ with edges $(w,c') \to (wb, \delta(c',b))$, rooted at (ε, c) (Fig. 3).

Theorem 8 (The step graph). *For every deterministic control machine (a DFA, or a DFA with a stack) and every configuration c: (1) P_c is an out-tree, hence a mangrove, and the configuration reached by a token is unique; (2) the step graphs of all configurations, joined under a universal source, form a mangrove, and a capped traversal from the source either computes the next configuration of every token in every context or reports that its budget is exceeded, never a partial table; (3) the vertices with exactly one way out form the* forced continuation *of c, the longest byte string every acceptable output from c begins with (the chains of Theorem 6); (4) for a forced string, the pre-token boundaries of a byte-pair tokenizer are cut vertices of its tokenization lattice, so the tokenizer's split is one choice per block (Theorem 7) and only the last block depends on bytes not yet known.*

Proof. (1) Every edge lengthens w, and a vertex (wb, c'') has the unique parent $(w, \delta^*(c,w))$, because the trie fixes w and determinism fixes the configuration. (2) Distinct roots give every vertex one path from the source, and Lemma 3 with unit budgets bounds the traversal. (3) A vertex with one out-edge has one allowed byte, which every acceptable continuation must take.

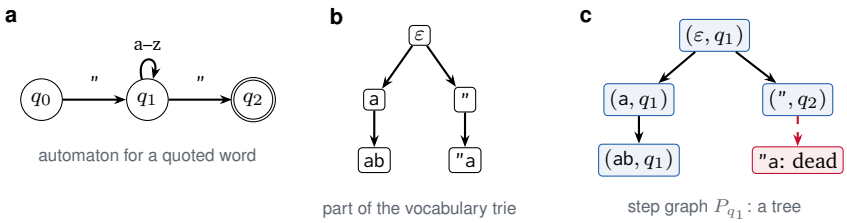


Fig. 3 | The step graph has exactly one path to every vertex. a, A three-state automaton for a quoted word. **b**, Part of the vocabulary trie. **c**, Their product from q_1: each trie node appears with the one state the automaton is in when it gets there. The token "a is dead from q_1 because no letter may follow the closing quote.

(4) A tokenizer that never merges across a pre-token boundary makes every tokenization pass through it, which is the definition of a cut vertex. □

3.3 Six mechanisms, and why each one works

Each mechanism is one of the proofs of Section 2 read as a recipe; we say what the problem is, what the engine does, why it is correct and what it measured. Supplementary Section D gives the algorithms and the formal guarantees.

1. One sweep builds the whole table. *Problem:* for every state and token the engine needs whether the token is allowed and where it leads. *What we do:* walk the trie once, byte depth by byte depth, with all states side by side in one array, so each level is one vectorized gather. *Why it works:* by Theorem 8(2) one traversal visits every (trie node, state) pair exactly once and stops cleanly if the grammar is too large. *Result:* 0.2–0.6 s per schema, the table is 3 bytes per (state, token), 77 MB for 104 states, which is the memory mechanism 3 removes.

2. Text the grammar forces is written for free. *Problem:* after `"name":"Maria"` the schema allows only `,"age":`, yet a normal decoder runs the model once per token for it. *What we do:* after each sampled token, look up the forced continuation, split it into tokens the way the tokenizer would, and feed the sampled and the forced tokens in one step (Fig. 4a). *Why it works:* forced text is the one-way-out chain of Theorem 8(3), certain before the model runs, a step of $m + 1$ tokens costs about one weight stream, the caches advance exactly as if the tokens had been sampled one by one, and by Theorem 8(4) committing every block but the last keeps the tokenizer's own split. *Result:* about 30% of a JSON output is forced, and a constrained stream runs at 180 against 138 tokens/s free on the 2B.

3. No table at all. *Problem:* the table grows with the grammar (1.5 GB at 1,970 states) and lives for the server's lifetime. *What we do:* keep only the DFA (512 bytes per state) and the vocabulary's bytes; each step a GPU kernel runs one thread per token, walks its bytes through the DFA and writes the state it reaches. *Why it works:* every token has one path (Theorem 8(1)), so the recomputed row equals the table's column, this is the ReachUL proof's discipline, recompute rather than store. *Result:* 7–18 μs per step against a 7–15 ms model step, and about 3 MB per grammar instead of 77 MB–1.5 GB, with identical outputs.

4. Recursive schemas with a small stack. *Problem:* a schema that refers to itself is not a DFA, the regex route unrolls three levels and then silently drops the recursive field. *What we do:* each recursive definition becomes its own DFA, a reference pushes a return state and jumps into the callee, and the callee's closing bracket pops, the 64-entry stack lives on the GPU next to the state. *Why it works:* this is the capped stack machine of Theorem 4, determinism is checked at compile time and the depth cap fails closed. *Result:* 170 tokens/s on the 2B, against 143 and 94 for llguidance with and without fast-forward.

5. Independent parts in parallel, certified. *Problem:* a decoder spends one weight stream per token even where the values of a JSON object do not depend on each other in the grammar. *What we do:* decode the values at once as rows of one batch, then run the model once over the assembled draft and ask, at every drafted token, whether it would have chosen exactly that token; where not, redo that value from the first wrong token (Fig. 4b). *Why it works:* the state after each key is a cut vertex (Theorem 7), so each value may start on its own, and the check is the very test sequential greedy decoding makes. *Result:* identical output with 374 instead of 672 weight streams (2B) and 484 instead of 686 (4B).

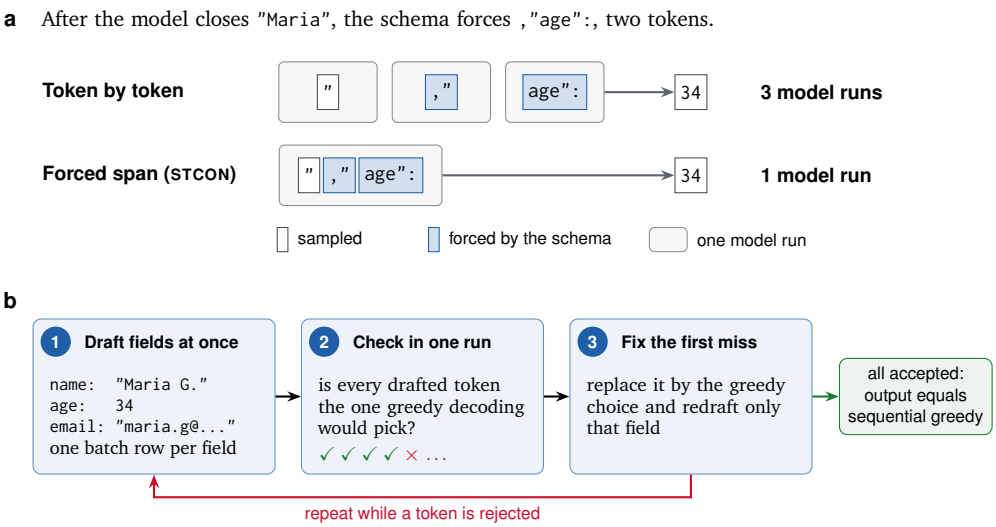


Fig. 4 | Two mechanisms that save weight streams. a, Forced spans: text the schema forces is fed in the run of the sampled token before it, and the model's next choice is unchanged. **b**, Certified parallel decoding: keys split an object into independent fields, all values are drafted at once, one run checks the draft, and the loop ends with exactly the sequential greedy output.

6. Valid under any token limit. *Problem:* an output that reaches the token limit inside a value does not parse. *What we do:* precompute for every state the fewest tokens needed to finish validly and allow a token only if a valid ending still fits. *Why it works:* this is the budget of Lemma 3 as a shortest-path invariant, and no less restrictive rule keeps every output valid (Theorem D.11). *Result:* 90 of 90 JSON outputs valid under every feasible limit, against 3 of 90.

3.4 The engine around the mechanisms

The mechanisms sit in a decode loop with three pieces of ordinary systems work (Supplementary Section E): 4-bit matrix–vector kernels tuned for 1–16 rows, so a few agents read each weight once; one fused kernel for the recurrent layer's step, and a loop that builds the next step while the current one runs, with the DFA state and the mask on the GPU, so the host is never waited for. The engine is built on Apple's MLX (Hannun et al., 2023). This engineering is held fixed on both sides of every comparison except the whole-system ones, which say so.

Takeaway

A constraint is not a tax on the model but information the engine reads off a one-path graph: what is certain is free, what is independent is parallel, what is needed each step is recomputed, and what recurses is a stack.

4 Results

All measurements are on one Apple M2 Pro (19-core GPU, 16 GB, wired GPU limit 10.7 GB) with Qwen3.5-2B (18 recurrent and 6 attention layers) and Qwen3.5-4B (24 and 8), both in 4-bit precision and greedy unless stated. Comparisons are made inside one process with interleaved rounds and medians (spread under 2%). llguidance (Moskal et al., 2024) and XGrammar (Dong et al., 2025) run as controllers inside our loop with the same kernels, so every difference is the control side alone; memory is read from MLX's own counters. Supplementary Sections F and H hold the full tables.

A constrained stream is faster than a free one (Theorem 6; mechanism 2). On the same prompts a schema-constrained stream decodes at 180 tokens/s where the free stream runs at 138 on the 2B, and at 82 against 63 on the 4B: $1.31\times$ on both models, because about 30% of the output tokens are forced chains fed without a weight stream. Against llguidance and XGrammar in the same loop (Table 3), STCON is the fastest engine that keeps the answer: $1.2\times$ llguidance's fast-forward at equal fidelity. XGrammar's jump-forward is about as fast but keeps only 0.65–0.71 of the answer's values against 0.87–0.94, because it re-tokenizes forced

Table 3 | Speed and quality against llguidance and XGrammar. One stream, decode tokens/s on the same prompts in the same loop with the same kernels, greedy. Fidelity is the share of the unconstrained answer's values kept (JSON schemas). For each library the better of its plain and overlapped integration. A separate run from the one quoted in the text (runs agree within about 5%).

	JSON, tok/s		Fidelity		Recursive, tok/s	
Engine	2B	4B	2B	4B	2B	4B
free decoding	137	66	—	—	135	67
STCON (ours)	**173**	85	**0.94**	0.87	161	84
llguidance, mask only	103	55	0.94	0.87	103	56
llguidance, fast-forward	143	73	0.94	0.87	141	73
XGrammar, mask only	103	56	0.92	**0.95**	103	55
XGrammar, jump-forward	159	**86**	0.65	0.71	**182**	**91**

Table 4 | Memory, from the ReachUL **walk kernel.** Top: constraint memory per grammar, and the memory each engine pays once per process. Bottom: one server process fed sixteen grammars in turn, prompt cache 50 MB.

Grammar	States	Token table	Walk kernel	llguidance	XGrammar
person (JSON schema)	104	77 MB	0.05 MB	<0.5 MB	0.18 MB
typed tool calls	1,106	824 MB	**0.57 MB**	fails	5.6 MB
96 fields	1,970	1.5 GB	1.0 MB	<0.5 MB	0.51 MB
recursive (solar)	—	38 MB	—	<0.5 MB	0.17 MB
sixteen grammars	—	7.2 GB	4.9 MB	4.7 MB	4.4 MB
shared per process	—	—	**2.9 MB**	153 MB	≈9 MB
Sixteen grammars in one server		**Grammars served**		**Above the model**	**Longest wait**
2B, token tables		15 of 16, then out of memory		8.2 GB	26 s
2B, walk kernel		**16 of 16**		**0.6 GB**	**2.8 s**
4B, token tables		14 of 16, then out of memory		6.9 GB	26 s
4B, walk kernel		**16 of 16**		**1.1 GB**	**6.6 s**

text across pre-token boundaries (Theorem 8(4)). Mask-only decoding with either library is 25% slower than free decoding although XGrammar's mask takes 3–13 μs: the cost is the CPU-to-GPU copy and the synchronous loop it forces.

Recursive schemas (Theorem 4; mechanism 4). The pushdown decodes recursive schemas at 170 tokens/s on the 2B, $1.19\times$ llguidance's fast-forward and $1.8\times$ its plain mode, and at 78.6 against 67.4 on the 4B, because forced spans cross the returns of the stack: after a node's last value the whole `]},{"name":"` continuation is one span. XGrammar's jump-forward is faster here, but its 4B trees are less than half as long as ours (155 against 382 tokens).

The table need not exist (Construction 2; mechanism 3). A token table costs $3|Q||\mathcal{V}|$ bytes and lives for the process. The walk kernel makes a grammar cost about 3 MB whatever its size, at the table's speed and with identical output (Table 4). The typed tool-call grammar, which fixes the name and type of every argument and has 1,106 states, needs an 824 MB table that takes 62–127 s to build; with the walk kernel it costs 3.4 MB, and on eight concurrent agent requests 46 of 46 calls have every argument present and of its declared type, against 43 of 52 with an untyped grammar. In a server that sees many grammars, tables kill the process at the fifteenth or sixteenth, the walk kernel serves all sixteen within 0.6–1.1 GB. XGrammar and llguidance also need only megabytes, the walk kernel matches them while keeping the mask on the GPU. XGrammar remains smaller on recursive schemas and compiles in milliseconds where our DFA takes up to 25 s.

Decisions, not tokens (Theorem 7; mechanism 5). Over 18 outputs the sequential decoder needs 672 weight streams on the 2B and the certified block decoder 374 ($1.80\times$ fewer); on the 4B, 686 against 484 ($1.42\times$). All 36 outputs are token-identical to the sequential ones. Wall time is not yet lower on this GPU, whose step is compute-bound from four rows, the count of weight streams is the hardware-independent number.

Valid under any limit (Lemma 3; mechanism 6). With the budget rule every JSON output is valid under every limit that admits a valid output at all, 90 of 90 against 3 of 90, and recursive schemas improve from 2 to 54 of 61 limits. As the limit tightens the engine closes lists early, then shortens strings, then empties them, instead of stopping mid-value (Fig. 7).

The whole engine. With our batched kernels added, sixteen coding agents that call tools in the model's native format, every reply constrained to a well-formed call, finish their suite in

29.3 s instead of 74.6 s with stock mlx-lm and llguidance, and more of them finish (12 against 9 of 16); eight concurrent tool-call requests through our OpenAI-compatible server are served at 410 against 134 tokens/s, every reply parsing (Supplementary Section G).

Takeaway

From the theorems: a constrained stream is $1.3\times$ faster than a free one and the fastest that keeps the answer; recursive schemas decode 1.2–$1.8\times$ faster than the best library mode that keeps them, a grammar costs 3 MB instead of up to 1.5 GB, so a server survives sixteen grammars; outputs stay valid under any feasible limit, and independent parts are decoded in parallel with 1.4–$1.8\times$ fewer weight streams and identical output.

5 Discussion and conclusion

The class memberships concern logarithmic memory and change nothing on a machine with gigabytes of it; what transfers is the object each proof constructs and the invariant that makes it correct. The unambiguous traversal exists because a few-path graph can be traversed without revisiting a configuration, the constraint's step graph has this property in its strongest form, $k = 1$, which is why one pass yields the table for every state, why forced output can be read off the graph, and why the table need not exist. The capped stack machine exists because a deterministic pushdown recognizes the language, applied to a schema's grammar it is the recursive constraint. The budget lemma, which every proof uses, is why every cap in the engine can be trusted. On a memory-bound machine the weight stream is the cost, and the theorems say exactly how many streams an output needs: its decisions.

Related work. Few-path logspace computation was introduced by Buntrock et al. (1991); Lange (1997) showed total reach-unambiguous reachability ReachUL-complete, Allender & Lange (1998) introduced mangroves and the $O(\log^2 n/\log\log n)$ simulation, and Garvin et al. (2014) proved ReachFewL $=$ ReachUL (see also Reinhardt & Allender, 2000; Pavan et al., 2010; Kannan et al., 2008; Dinesh & Sasidharan, 2026). For decoding, SynCode and grammar-constrained decoding handle context-free grammars (Ugare et al., 2024; Geng et al., 2023), SGLang decodes runs of forced transitions in one step (Zheng et al., 2024), Koo et al. (2024) compose the tokenizer with the automaton, and non-canonical tokenizations are known to hurt accuracy (Beurer-Kellner et al., 2024; Geh et al., 2024; Vieira et al., 2024). Our forced spans are speculative drafts (Leviathan et al., 2023; Chen et al., 2023; Fu et al., 2024) accepted with certainty.

Limitations and open problems. The theory applies where inference is a deterministic machine, the dense arithmetic is not one, and ideas that treated it as such were measured and do not pay (Supplementary Section J), which is also why certified blocks save streams and not yet time on this GPU. The measurements use one machine and one model family. On the theory side, three questions remain: whether the problem for $k = 1$ is in L; whether simple paths can replace walks, and whether the method extends from few computation prefixes to few accepting computations, which would place $\textsc{stcon}_{\text{sf}}$ in StrongFewL.

© Generated Computers 2026

Data and code availability

All prompts, schemas, grammars and result files behind every table ship with the paper, each number tagged with the script that produced it. The engine are at https://github.com/Generated-Computers/Locals.

Acknowledgements

The author thanks Simon Apers and Roman Edenhofer for posing the question so precisely.

Statement of novelty

The theoretical results were obtained by the author in August 2026. The remaining time went into the application: identifying the few-path structure of constrained decoding, deriving the mechanisms and their guarantees from the proofs, and building the engine.

References

Allender, E. and Lange, K.-J. RUSPACE$(\log n) \subseteq DSPACE(\log^2 n/\log\log n)$. *Theory of Computing Systems*, 31(5):539–550, 1998.

Allender, E. and Ogihara, M. Relationships among PL, #L, and the determinant. *RAIRO Theoretical Informatics and Applications*, 30:1–21, 1996.

Apers, S. and Edenhofer, R. Directed st-connectivity with few paths is in quantum logspace. In *40th Computational Complexity Conference (CCC), LIPIcs 339, Article 18*, 2025.

Beigel, R. and Fu, B. Circuits over PP and PL. In *IEEE Conference on Computational Complexity*, pp. 24–35, 1997.

Beurer-Kellner, L., Fischer, M., and Vechev, M. Guiding LLMs the right way: fast, non-invasive constrained generation. In *ICML*, 2024.

Buntrock, G., Jenner, B., Lange, K.-J., and Rossmanith, P. Unambiguity and fewness for logarithmic space. In *Fundamentals of Computation Theory (FCT), LNCS 529*, pp. 168–179, 1991.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.

Cook, S. A. Deterministic CFL's are accepted simultaneously in polynomial time and log squared space. In *STOC*, pp. 338–345, 1979.

Dinesh, K. and Sasidharan, C. Space complexity of reachability in simple path graphs. In *MFCS, LIPIcs 386, Article 87*, 2026.

Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., and Chen, T. XGrammar: flexible and efficient structured generation engine for large language models. In *MLSys*, 2025.

Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of LLM inference using lookahead decoding. In *ICML*, 2024.

Garvin, B., Stolee, D., Tewari, R., and Vinodchandran, N. V. ReachFewL = ReachUL. *Computational Complexity*, 23(1):85–98, 2014.

Geh, R. L., Zhang, H., Ahmed, K., Wang, B., and Van den Broeck, G. Where is the signal in tokenization space? In *EMNLP*, 2024.

Geng, S., Josifoski, M., Peyrard, M., and West, R. Grammar-constrained decoding for structured NLP tasks without finetuning. In *EMNLP*, 2023.

Hannun, A., Digani, J., Katharopoulos, A., and Collobert, R. MLX: efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

Hoang, T. M. and Thierauf, T. On closure properties of GapL. Technical Report TR04-024, ECCC, 2004.

Immerman, N. Nondeterministic space is closed under complementation. *SIAM Journal on Computing*, 17(5):935–938, 1988.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. LiveCodeBench: holistic and contamination free evaluation of large language models for code. *arXiv:2403.07974*, 2024.

Kannan, S., Khanna, S., and Roy, S. STCON in directed unique-path graphs. In *FSTTCS, LIPIcs 2*, pp. 256–267, 2008.

Koo, T., Liu, F., and He, L. Automata-based constraints for language model decoding. In *COLM*, 2024.

Lange, K.-J. An unambiguous class possessing a complete set. In *STACS, LNCS 1200*, pp. 339–350, 1997.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

Moskal, M. et al. llguidance: super-fast structured outputs. https://github.com/guidance-ai/llguidance, 2024.

Pavan, A., Tewari, R., and Vinodchandran, N. V. On the power of unambiguity in log-space. *arXiv:1001.2034*, 2010.

Reingold, O. Undirected connectivity in log-space. *Journal of the ACM*, 55(4), 2008.

Reinhardt, K. and Allender, E. Making nondeterminism unambiguous. *SIAM Journal on Computing*, 29(4):1118–1131, 2000.

Sudborough, I. H. On deterministic context-free languages, multihead automata, and the power of an auxiliary pushdown store. In *STOC*, pp. 141–148, 1976.

Sudborough, I. H. On the tape complexity of deterministic context-free languages. *Journal of the ACM*, 25(3):405–414, 1978.

Szelepcsényi, R. The method of forced enumeration for nondeterministic automata. *Acta Informatica*, 26:279–284, 1988.

Ugare, S., Suresh, T., Kang, H., Misailovic, S., and Singh, G. SynCode: LLM generation with grammar augmentation. *arXiv:2403.01632*, 2024.

Vieira, T., LeBrun, B., Giulianelli, M., Gastaldi, J. L., DuSell, B., Terilla, J., O'Donnell, T. J., and Cotterell, R. From language models over tokens to language models over characters. *arXiv:2412.03719*, 2024.

Willard, B. T. and Louf, R. Efficient guided generation for large language models. *arXiv:2307.09702*, 2023.

Yang, S., Kautz, J., and Hatamizadeh, A. Gated Delta networks: improving Mamba2 with delta rule. In *ICLR*, 2025.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: efficient execution of structured language model programs. In *NeurIPS*, 2024.

Supplementary Information

A Background in plain words, and the problems measured **13**
B Proofs of the main theorem **13**
B.1 Machines, classes and the published facts we use 13
B.2 The universal source and a worked example 14
B.3 Construction 1 in full 14
B.4 Construction 2 in full 16
B.5 Membership in ReachUL without an oracle 17
B.6 Construction 3 in full 18
B.7 The main theorem 19
C Further results: budgets, completeness, thresholds and structure **19**
C.1 Statements 19
C.2 Proofs: budgets, compiler and completeness 20
C.3 Proofs: the threshold phase transition and the StrongFewL boundary 21
C.4 Proofs of the structure theorems 21
D The mechanisms in detail **22**
D.1 Cost model, control machines and the step graph 22
D.2 The token table, with and without storing it 23
D.3 Forced output costs no pass 23
D.4 Splitting forced text the way the tokenizer would 24
D.5 Valid outputs under a token budget 24
D.6 Recursion: a deterministic pushdown with a capped stack . . 25
D.7 Independent parts in parallel, certified 25
D.8 The reply grammar for tool calls 26
E Implementation **26**
E.1 Hardware, models and protocol 26
E.2 Constraint compilation 26
E.3 The walk kernel 27
E.4 The forced planner 27
E.5 The pushdown constraint 27
E.6 Certified block-parallel decoding 27
E.7 Rows on demand 27
E.8 Engine: kernels, pipelining and batching 27
E.9 Server, reply grammar and agents 28
E.10 Exactness and tests 28
F Speed: all measurements **28**
F.1 Speed of a single constrained stream 28
F.2 Certified blocks 29
G The whole engine: throughput, agents and LiveCodeBench **29**
H Memory: all measurements **31**
I Budgets, answer quality and exactness **33**
I.1 Budget masking 33
I.2 Answer quality 33
J What did not help **34**

A Background in plain words, and the problems measured

Decoding. A language model is given a prompt, a list of tokens, and produces a score for every token of its vocabulary (248,320 tokens for Qwen3.5) as the next one. The engine picks one, appends it and asks again. Taking the best token every time is greedy decoding, drawing at random from the scores is sampling. Processing the prompt is the prefill; producing the answer token by token is the decode. Each decode step is one weight stream.

Caches. A transformer keeps, for every token it has seen, a small vector per layer (the KV cache), so that it need not re-read the prompt. The Qwen3.5 models are hybrid: three of every four layers are recurrent (Gated DeltaNet; Yang et al., 2025) and keep a fixed-size state. On the 4B the KV cache costs 32 KB per token per stream and the recurrent state 50 MB per stream.

Constraints. A constraint (a JSON schema, a regular expression, the grammar of tool calls) is compiled into an automaton that reads one byte at a time, at each state some bytes are allowed. The engine forbids at every step the tokens whose bytes the automaton would reject (the mask) and moves the automaton along the bytes of the chosen token.

Streams and batches. An answer being decoded, with its prompt, caches and automaton state, is a stream. Several streams decoded together form a batch, and one step serves all of them per weight stream. In an agent system the streams are agents waiting for answers, the engine's slots are the largest batch it will run.

What goes wrong as the agents multiply. Table 5 lists the three problems we measured when constraints are enforced the usual way; everything in it is measured in Supplementary Sections F and H.

Table 5 | Three things that go wrong on a laptop as grammars and agents multiply. One Apple M2 Pro, 16 GB, Qwen3.5-2B and 4B in 4-bit precision.

Problem	What we measured	Where it comes from
Every grammar costs a table	77 MB (104 states) to 1.5 GB (1,970 states), a typed tool grammar 824 MB and 62–127 s to build, tables live as long as the process	the table is vocabulary × states, and the vocabulary has 248k tokens
A server dies as grammars accumulate	16 schemas in one server: 6.9 GB above the model at the 14th (4B), a new grammar's first request waits 26 s, throughput collapses to 11 tokens/s, the 15th kills the process	tables plus the model no longer fit in the wired GPU limit (10.7 GB)
Enforcing the grammar costs speed	with masks only, llguidance and XGrammar decode at 103 tokens/s against 137 free (2B); XGrammar's jump-forward recovers the speed at a cost in the answer (fidelity 0.65)	the mask is computed on the host and copied to the GPU every step

B Proofs of the main theorem

B.1 Machines, classes and the published facts we use

A nondeterministic logspace machine has a read-only input tape and a work tape of $O(\log M)$ cells. A configuration is the state, the input head position and the work tape with its head; $G_M(x)$ is the configuration graph on input x and C_0 the start configuration. We assume a clock, so every path halts in polynomial time and $G_M(x)$ is acyclic, and a unique accepting configuration. A path halts by entering a configuration with no successor. Halted paths are *not* merged into one rejecting configuration, a failed guess simply ends where it is.

- M is *reach-unambiguous* if on every input every configuration has at most one path from C_0. ReachUL is the class of languages accepted by such machines; ReachFewL allows polynomially many paths to every configuration, and ReachFewL = ReachUL (Garvin et al., 2014). UL asks only for at most one accepting path. StrongFewL asks for polynomially many paths between *every* ordered pair of configurations, including pairs unreachable from C_0 (Buntrock et al., 1991).
- A deterministic auxiliary pushdown automaton has a two-way read-only input, an $O(\log M)$ work tape and an unbounded stack not charged to the space bound. Polynomial-time ones accept exactly LOGDCFL (Sudborough, 1976, 1978), and LOGDCFL $\subseteq$ SC$^2 = \bigcup_c$ DTISP$(M^c, O(\log^2 M))$ by Cook (1979); one machine meets both bounds.
- #L counts accepting paths of an NL machine; GapL is the difference of two #L functions and is closed under logspace-uniform polynomial sums and products (Allender & Ogihara, 1996; Hoang & Thierauf, 2004). C$_=$L $= \{x : g(x) = 0\}$ and PL $= \{x : g(x) > 0\}$ for $g \in$ GapL.

We use three published facts. (a) The total language $\textsc{stcon}_{\mathrm{ru}} = \{\langle J, r, t\rangle : N_J(r, x) \le 1\ \forall x,\ N_J(r, t) = 1\}$ is ReachUL-complete (Lange, 1997). (b) ReachUL is closed under complement and under deterministic logspace oracle computations with logspace-generated queries: L^{ReachUL} = ReachUL (Garvin et al., 2014). (c) NL = coNL (Immerman, 1988; Szelepcsényi, 1988). Supplementary Section B.5 gives a second proof of the ReachUL membership that uses neither (a) nor (b).

Budget languages. In RootBudget the input is a graph $H = (Q, E)$ with a root r, a set F of accepting vertices and a budget $\kappa : Q \to \mathbb{N}$ with total $B = \sum_q \kappa(q)$, the answer is yes when $N_H(r, q) \le \kappa(q)$ for all q and $N_H(r, f) \ge 1$ for some $f \in F$. In PairBudget budgets $\kappa(u, v)$ are given for all pairs, the answer is yes when $N_G(u, v) \le \kappa(u, v)$ for all pairs and $N_G(s, t) \ge 1$. Budgets are written in unary, or in binary with an

explicitly checked bound $B \le M^c$ (a first scan rejects violations, so this is a checkable input format, not a promise). $\textsc{stcon}_{\mathrm{sf}}$ is PairBudget with $\kappa \equiv k$, and by Lemma B.1 it is RootBudget on $U(G)$ with budget 1 at ρ, $\kappa(a,v)$ at (a,v), target (s,t) and total budget $1 + n^2k$.

B.2 The universal source and a worked example

Lemma B.1 (Universal source with copies). *Let $U(G)$ have a root ρ, a vertex (a,v) for every pair, an edge $\rho \to (a,a)$ for every a, and an edge $(a,u) \to (a,v)$ for every edge $u \to v$ of G and every tag a. Then $N_{U(G)}(\rho,(a,v)) = N_G(a,v)$ for all a, v.*

Proof. A walk from ρ to (a,v) starts with the only edge $\rho \to (a,a)$ and then stays in copy a, whose tag never changes, dropping the first edge is a bijection onto the walks from a to v in G, and the edge $\rho \to (a,a)$ alone is the empty walk. □

Without the copies a universal source would bound only the sums $\sum_a N(a,v)$, the copies keep the condition on every pair. Table 6 runs the three constructions on the running examples of Fig. 1, and Fig. 5 shows the lift of Construction 2.

Table 6 | The three constructions on the running examples ($k = 2$, $n = 5$, per-source cap $nk = 10$). Each accepts $G_\star$ and rejects $G_\circ$, each for its own reason.

Construction	$G_\star$ (yes-instance)	$G_\circ$ (no-instance)
1. Capped stack	from s: seven walks, below the cap; recount at t gives $2 \le k$	from v: one node per length; node 11 hits the cap
2. Certified lift	$\alpha = 6$ fails certification (two root paths to $(v,2,2)$); $\alpha = 0$ certifies, two states above v and above t	from v: one walk per length, so every α certifies; six states above v exceed the budget 2
3. Zero test	$f_{st} = f_{sv} = 2$, all others ≤ 1; every product vanishes, $g = 0$	$f_{vv} = 6$, $P_{vv} = 120$; $f_{st} = 8$, $P_{st} = 42$; $g > 0$

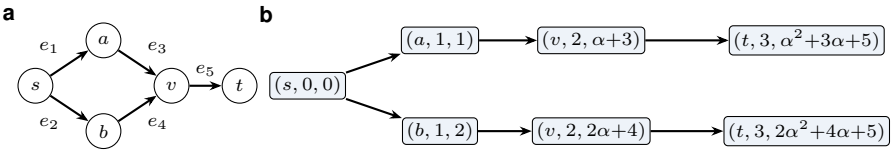


Fig. 5 | The lift names every walk. a, $G_\star$ with numbered edges. **b**, The lift H_α: a state is (vertex, length, hash) and each walk lifts to one root path. Over $\mathbb{F}_7$ the two hashes above v coincide only at $\alpha = 6$, where certification fails, at every other α there are exactly $2 = N(s,v)$ states above v.

B.3 Construction 1 in full

Lemma B.2 (Unfolding trees). *Let $|T_a^L| = \sum_b N^{\le L}(a,b)$ be the number of walks of length at most L from a. (i) If $|T_a^{nk}| > nk$ then G is not k-strongly few. (ii) If G is k-strongly few then $|T_a^{nk}| \le nk$ for every a. (iii) If G has a cycle, $|T_a^{nk}| \ge nk + 1$ for every vertex a on it. (iv) If G is acyclic, $N^{\le nk}(a,b) = N(a,b)$.*

Proof. (i) More than nk nodes among n endpoints put at least $k+1$ distinct walks on some endpoint. (ii) is (i) read backwards. (iii) Going around the cycle gives a walk of every length $0, \dots, nk$. (iv) In an acyclic graph every walk is a simple path, of length at most $n - 1 \le nk$. □

Lemma B.3 (Stack invariant). *At every moment of Algorithm 1 the stack holds exactly the frames of the ancestors of the current node, from the root down to the parent, and each frame stores the index of the first successor not yet tried.*

Proof. True at the start. A descent advances the parent's successor index and pushes the resulting frame, a return pops exactly that frame, there are no other stack operations. □

Proof of Theorem 4. *Correctness.* Let the input be in $\textsc{stcon}_{\mathrm{sf}}$. A cycle would give infinitely many walks, so the graph is acyclic; by Lemma B.2(ii) every first-pass traversal finishes, by (iv) every second-pass count is the exact value $N(a,b) \le k$, and *reached* becomes true exactly when $N(s,t) \ge 1$. Conversely, if the algorithm accepts, the first pass never overflowed, so by (iii) there is no cycle, then all counts are exact by (iv), every count is at most k and the (s,t) count is positive. The counts start only after the first pass has bounded every tree by nk nodes, so a bad input cannot cause an exponentially long count.

Resources. The work tape holds $a, b, cur, next, depth, nk, total, c_b$ and loop counters: $O(\log M)$ bits. A frame (v,j) with $v \in \{1..n\}$, $j \in \{1..n+1\}$ is written over $\{0,1,\#,\bot\}$ as $\#\,\mathrm{bin}_w(v)\,\mathrm{bin}_w(j)$ with $w = \lceil \log_2(n+2) \rceil$, $\bot$ marking the bottom, the current frame sits in a work-tape buffer and is pushed symbol by symbol. Until a traversal ends, the stack holds at most nk frames. A traversal enters at most $nk + 1$ nodes and

Algorithm 1 TRAVERSE(a, b): capped traversal from source a with a budget of nk nodes; in target mode it also counts the nodes that end at b.

$cur \leftarrow a$; $next \leftarrow 1$; $depth \leftarrow 0$; $total \leftarrow 1$; $c_b \leftarrow [a = b]$
loop
  if $depth = nk$ **or** $next > n$ **then** `// nothing left here: go up`
    if stack empty **then return** $(\text{OK}, total, c_b)$
    end if
    pop (v, j); $cur \leftarrow v$; $next \leftarrow j$; $depth \leftarrow depth - 1$; **continue**
  end if
  $w \leftarrow next$; $next \leftarrow next + 1$
  if $(cur, w) \notin E$ **then continue**
  end if
  push $(cur, next)$; $cur \leftarrow w$; $next \leftarrow 1$; $depth{+}{+}$; $total{+}{+}$
  if $total = nk + 1$ **then return** OVERFLOW `// the cap is hit`
  end if
  if target mode **and** $cur = b$ **then**
    $c_b \leftarrow c_b + 1$; **if** $c_b = k + 1$ **return** OVERFLOW
  end if
end loop

Algorithm 2 The decider for STCON_{sf} (no promise).

validate the input; reject if malformed or $k = 0$; $reached \leftarrow$ **false**
for every source a **do** `// pass 1: every tree is small`
  if TRAVERSE$(a,$ no target$)$ overflows **then reject**
  end if
end for
for every source a and target b **do** `// pass 2: exact counts`
  if TRAVERSE(a, b) overflows **then reject**
  end if
  if $a = s$, $b = t$ and $c_b \geq 1$ **then** $reached \leftarrow$ **true**
  end if
end for
accept iff $reached$

tries at most n successors at each; even at $O(M)$ per edge test this is $O(n^2kM + nk \log M)$ steps, and there are $n + n^2$ traversals. So the decider is a polynomial-time deterministic auxiliary pushdown automaton with a logarithmic work tape, which places the language in LOGDCFL (Sudborough, 1978), and LOGDCFL $\subseteq$ SC^2 (Cook, 1979). □

Remark B.4 (Multigraphs). The traversal works for a multigraph with m edges if it enumerates outgoing edges rather than successor vertices, the continuation index then takes $\lceil \log_2(m+2) \rceil$ bits while the vertex field keeps $\lceil \log_2(n+2) \rceil$. Parallel edges give different walks and cannot be merged. The cap can also be placed on the number of nodes alone: traverse the full unfolding and reject on entering node $B+1$ (Theorem C.10).

B.4 Construction 2 in full

Take a RootBudget input (H, r, κ, F) with edge occurrences $e_1, \dots, e_m$ and total budget $B \geq 1$ (reject $B = 0$ or $\kappa(r) = 0$ at once). Let $S = \sum_v \binom{\kappa(v)}{2}$ and let p be the least prime with $p > \max\{m, BS, 2\}$, found by trial division; p is polynomial by Bertrand's postulate and depends only on the written budgets, not on whether they hold.

Definition B.5 (The lift). H_α has states (v, d, z) with $0 \leq d \leq B$ and $z \in \mathbb{F}_p$, an edge $(u, d, z) \to (v, d+1, \alpha z + i \bmod p)$ for every edge $e_i = (u, v)$ and $d < B$, and root $(r, 0, 0)$. It has $|Q|(B+1)p$ states and a logspace adjacency predicate.

Lemma B.6 (Separation). *If all budgets hold, some $\alpha \in \mathbb{F}_p$ gives distinct hashes to any two distinct walks of the same length and endpoint.*

Proof. For two distinct walks of the same length $d \leq B$, the difference of their hashes is a polynomial in α of degree at most $B-1$, nonzero because some edge index differs and indices are below p; so the pair collides at most at $B-1$ points. With the budgets holding, at most $\kappa(v)$ walks end at v, so there are at most S relevant pairs and at most $(B-1)S < p$ bad points. □

Lemma B.7 (Exact fibers). *Walks of length $d \leq B$ from r to v correspond bijectively to root paths ending at $(v, d, h_\alpha(P))$. Whenever H_α is reach-unambiguous from its root, whether or not the budgets hold, $N_H^{\leq B}(r, v) = \big|\{(d, z) : (r, 0, 0) \rightsquigarrow (v, d, z) \text{ in } H_\alpha\}\big|$.*

Proof. Along a lifted path the hash is forced by the recurrence, so every walk lifts to exactly one path and projection inverts lifting. Two distinct walks reaching the same state would give it two root paths. □

Proof of Theorem 5. The controller enumerates $\alpha \in \mathbb{F}_p$ and queries $\textsc{stcon}_{\mathrm{ru}}(H_\alpha, (r,0,0), (r,0,0))$, which holds exactly when H_α is reach-unambiguous from its root (the only path from the root to itself is the empty one). For the first certified α it scans every fiber, querying reachability of each lifted state with a counter saturated at $\kappa(v)+1$; it rejects if a count exceeds its budget, and otherwise accepts iff some fiber of a vertex in F is nonempty. If no α is certified it rejects.

Correctness. On a valid input Lemma B.6 gives a certified α, every fiber count equals $N^{\leq B}(r, v) \leq \kappa(v)$ by Lemma B.7, and reachability is read off correctly. On an invalid input either no lift is certified, or a certified lift gives exact bounded counts and by Lemma 3 some count exceeds its budget, or the reachability condition fails. To check budgets alone take $F = \{r\}$.

Resources. The controller stores α, an endpoint, a layer, a residue and a saturated counter and makes polynomially many queries: it is a deterministic logspace oracle machine with oracle $\textsc{stcon}_{\mathrm{ru}} \in$ ReachUL, so the language is in ReachUL by fact (b). With Lemma B.1, PairBudget and $\textsc{stcon}_{\mathrm{sf}}$ are in ReachUL. Closure under complement gives UL $\cap$ coUL, and ReachUL $=$ RUSPACE$(\log n) \subseteq$ DSPACE$(\log^2 n / \log\log n)$ gives the space bound (Allender & Lange, 1998), a pure space bound with no time claim. □

The oracle composition. The identity $\mathsf{L}^{\mathsf{ReachUL}} = \mathsf{ReachUL}$ is realized as follows. At each query the machine guesses the answer bit and simulates a reach-unambiguous recognizer for the oracle language or its complement, as the bit says. Only an accepting sub-computation returns, the returning path is unique and carries the correct bit, and all other branches halt without merging. Throughout a sub-computation the machine keeps the answer bit, the saved controller configuration and the query index, so different queries and different answer branches have different configurations; local data may be erased only on the unique returning branch. So every reachable configuration of the combined machine has at most one history. The query need not be stored: its symbols are regenerated from a short description and a position.

Remark B.8 (The diamond, in numbers). Take $r \to a \to v$, $r \to b \to v$ with edge numbers $1, 3$ on the upper route and $2, 4$ on the lower one. The hashes of the two walks to v are $\alpha + 3$ and $2\alpha + 4$; over $\mathbb{F}_7$ they coincide at $\alpha = 6$, and that lift fails certification. With budgets $(1, 1, 1, 2)$ at (r, a, b, v) the certified lift gives exactly two states above v and the input is accepted. Lowering the budget at v to one does not stop lifts from passing, but the exact count two rejects it. The existence of an unambiguous lift alone is therefore not the acceptance criterion; the budget comparison is needed.

B.5 Membership in ReachUL without an oracle

Here we prove $\textsc{stcon}_{\mathrm{sf}} \in \mathsf{ReachUL}$ without $\textsc{stcon}_{\mathrm{ru}}$ and without $\mathsf{L}^{\mathsf{ReachUL}} = \mathsf{ReachUL}$, hashing vertex sequences instead of edge numbers. Let the input be $\langle G, s, t, 1^k\rangle$ with $V = \{0, \dots, n-1\}$. Put $L = nk$ and let p be the least prime with $p > \max(n, L(nk)^2)$, found in logspace by trial division; all numbers below have $O(\log M)$ bits. For a walk $\omega = (v_0, \dots, v_i)$ put $H_\alpha(\omega) = \sum_{j=0}^{i} v_j \alpha^{i-j} \bmod p$, computed by $h_0 = v_0$, $h_j = \alpha h_{j-1} + v_j$, and write $N_i(a, v)$ for the number of walks of length exactly i.

Definition B.9 (Hashed unfolding). For $a \in V$ and $\alpha \in \mathbb{F}_p$, $\Gamma_{a,\alpha}$ has states $(v, i, h) \in V \times \{0..L\} \times \mathbb{F}_p$, edges $(u, i, h) \to (v, i+1, \alpha h + v)$ for $(u, v) \in E$ and $i < L$, and root $r_a = (a, 0, a)$. Layer i is the set of states with second coordinate i. $\Gamma_{a,\alpha}$ is *certified through layer d* if every state of layers $0..d$ has at most one path from r_a.

Lemma B.10 (Walks are root paths). *$\omega \mapsto ((v_0, 0, h_0), \dots, (v_i, i, h_i))$ is a bijection from walks of length $i \le L$ from a onto paths of length i from r_a, ending at $(v_i, i, H_\alpha(\omega))$. Hence $\Gamma_{a,\alpha}$ is certified through layer d iff H_α separates any two walks of the same length $i \le d$ from a to the same vertex, and then $N_i(a, v) = |\{h : (v, i, h)$ reachable from $r_a\}|$.*

Proof. Edges project to edges of G, and along a path the third coordinate is determined by the first through Horner's rule; conversely every walk lifts to exactly one path. □

Lemma B.11 (A good evaluation point exists). *If $\sum_v N^{\le L}(a, v) \le nk$, fewer than p values of α fail to certify $\Gamma_{a,\alpha}$ through layer L.*

Proof. Two distinct walks of the same length $i \le L$ from a to v have different vertex sequences with entries below $n < p$, so their difference polynomial is nonzero of degree at most L and has at most L roots. There are at most $(nk)^2$ such pairs, so at most $L(nk)^2 < p$ bad values. □

Algorithm 3 $\textsc{Tour}(a, \alpha, d)$: Euler tour of the reachable states of layers $\le d$ of $\Gamma_{a,\alpha}$, assumed certified through layer d, calling $\textsc{Visit}$ once per state.

```
x ← r_a; VISIT(x); mode ← down
loop
    if mode = down then
        if layer(x) < d and x has an out-neighbour then
            x ← first out-neighbour of x; VISIT(x); continue
        else mode ← up
        end if
    end if
    if x = r_a then return
    end if
    u ← PARENT(x) // nondeterministic
    if u has an out-neighbour after x then
        x ← that out-neighbour; VISIT(x); mode ← down
    else x ← u
    end if
end loop
```

$\textsc{Parent}(x)$ for x at layer $j \ge 1$: guess a state u of layer $j-1$ with $u \to x$; guess $z_0 = r_a, z_1, \dots, z_{j-1}$, each an out-neighbour of the previous, storing only z_m and m; if $z_{j-1} = u$ return u, otherwise halt (this path dies).

Lemma B.12 (Touring a certified prefix). *Let $\Gamma = \Gamma_{a,\alpha}$ be certified through layer d and let T be the set of states of layers $\le d$ reachable from r_a. (i) T with the edges of Γ is an out-tree rooted at r_a in which the children of every $x \in T$ of layer $< d$ are exactly its out-neighbours. (ii) Algorithm 3, started in a configuration with a unique path into it, visits every state of T once, has exactly one returning computation path, reaches every configuration on its way by exactly one path, and uses $O(\log M)$ work space beyond that of $\textsc{Visit}$.*

Proof. (i) Every $x \in T$ has exactly one root path; its predecessor is its only in-neighbour in T, since another $u \in T$ would give a second path $r_a \rightsquigarrow u \to x$. Out-neighbours of $x \in T$ with $\mathrm{layer}(x) < d$ are reachable, hence in T. (ii) By (i) the tour is the deterministic Euler tour of a tree, provided $\textsc{Parent}$ returns the parent. In $\textsc{Parent}(x)$ the guessed z_m follow edges from r_a, so each lies in T and its guessed prefix is its unique path, the guess succeeds iff u is reachable at layer $j-1$ and the guessed path is its path, and by (i) the only reachable in-neighbour of x is its parent. So exactly one branch succeeds. Tour configurations $(x, mode, \text{pending child})$ occur in a fixed order without repetition, a recovery configuration $(x, mode, u, z_m, m)$ arises only from the one tour configuration that calls $\textsc{Parent}(x)$, by one choice of u and guesses determined by z_m, so it too has one path. The work tape holds x, u, z_m, m and the mode: $O(\log(nLp)) = O(\log M)$ bits. □

Lemma B.13 (Certifying layer by layer). *CERTIFY(a,α) runs, for $d=0,\dots,L-1$ and every state y of layer $d+1$, TOUR(a,α,d), counting up to 2 the visited states x of layer d with $x\to y$, and fails as soon as a count reaches 2. It returns OK iff $\Gamma_{a,\alpha}$ is certified through layer L, calls TOUR only on certified prefixes, has exactly one returning path and reaches every configuration by exactly one path.*

Proof. Layer 0 is $\{r_a\}$. If the graph is certified through layer d, the number of paths into y at layer $d+1$ equals its number of reachable in-neighbours, which the tour counts, all counts are at most one exactly when layer $d+1$ is certified. The loop variables d, y and the count are part of every configuration, so by Lemma B.12 different calls share no configuration. □

Algorithm 4 The oracle-free ReachUL decider for STCON_{sf}.

```
compute n, L = nk and the prime p; R ← false
for a = 0, ..., n − 1 do
    ok ← false
    for α = 0, ..., p − 1 do
        if CERTIFY(a, α) = OK then
            for v = 0, ..., n − 1 do
                C ← states visited by TOUR(a, α, L) whose vertex is v, counted up to k + 1
                if C > k then reject
                end if
                if a = s, v = t and C ≥ 1 then R ← true
                end if
            end for
            ok ← true; break
        end if
    end for
    if not ok then reject
    end if
end for
if R accept else reject
```

Theorem B.14. *Algorithm 4 is a reach-unambiguous logspace machine with polynomial running time that accepts exactly STCON_{sf}.*

Proof. *Correctness.* For a certified α the count for v is $\sum_{i\le L} N_i(a,v) = N^{\le L}(a,v)$, saturated at $k+1$, by Lemma B.10. On a yes-instance, $\sum_v N^{\le L}(a,v) \le nk$ for every a, Lemma B.11 gives a certified α, all counts are at most k, and R becomes true. If the algorithm accepts, then for every a some α certified and $N^{\le L}(a,v)\le k$ for all v; by Lemma 3 with $\kappa\equiv k$ and $B=nk=L$, $N(a,v)\le k$ for all v, and $N(s,t)\ge N^{\le L}(s,t)\ge 1$. Rejections are correct by the contrapositive of Lemma B.11 and because $N\ge N^{\le L}$. *Unambiguity.* The outer control is deterministic; every nondeterministic step happens inside PARENT, within tours of certified prefixes (Lemma B.13; for the counting tours CERTIFY has just returned OK). Each call has one returning branch and reaches every configuration by one path (Lemma B.12), and the outer variables (a,α,v,C,R,ok) change strictly between calls and are part of every configuration. *Resources.* A constant number of $O(\log M)$-bit numbers; polynomially many tours, each visiting at most $n(L+1)p$ states with $O(L)$ steps per parent recovery. □

The procedure CERTIFY is itself a reach-unambiguous decider for the unambiguity of a layered graph through a given layer, that is what makes the proof self-contained.

B.6 Construction 3 in full

For $(u,v)\neq(s,t)$ put $P_{uv}=\prod_{j=0}^{k}(f_{uv}-j)$, for the designated pair $P_{st}=\prod_{j=1}^{k}(f_{st}-j)$, and $g=P_{st}^2+\sum_{(u,v)\neq(s,t)}P_{uv}^2$.

Theorem B.15 (Signed counting). *$g\in$ GapL, and $g(x)=0$ if and only if $x\in\text{STCON}_{\text{sf}}$. Hence $\text{STCON}_{\text{sf}}\in$ C$_=$L; likewise* PAIRBUDGET, ROOTBUDGET $\in$ C$_=$L *for budgets of polynomial total size.*

Proof of Theorem B.15. For $x=\langle G,s,t,1^k\rangle$ let $f_{uv}(x)=N_G^{\le nk}(u,v)$. A nondeterministic logspace machine that chooses a length at most nk and then that many edge codes, accepting iff they form a walk from u to v, has one accepting computation per walk, so $f_{uv}\in\#$L. GapL is closed under logspace-uniform polynomial sums and products: for a sum the machine guesses a summand's index, for a product it simulates the factors one after another keeping only a sign bit and the current index. Hence $g\in$ GapL. Every term of g is an integer square, so $g(x)=0$ exactly when all truncated counts are at most k and the designated count is

positive, and by Lemmas B.1 and 3 exactly when $x \in \textsc{stcon}_{\mathrm{sf}}$. Malformed inputs and $k = 0$ get a nonzero constant. □

Budget versions. For pair budgets let $B_u = \sum_v \kappa(u,v)$, $L = \max_u B_u$, $f_{uv} = N^{\le L}(u,v)$, $P_{uv} = \prod_{j=0}^{\kappa(u,v)}(f_{uv} - j)$ for ordinary pairs and $P_{st} = \prod_{j=1}^{\kappa(s,t)}(f_{st} - j)$ (reject at once if $\kappa(s,t) = 0$). For root budgets let $f_v = N_H^{\le B}(r,v)$, $P_v = \prod_{j=0}^{\kappa(v)}(f_v - j)$, $f_F = \sum_{v \in F} f_v$ and $P_F = \prod_{j=1}^{B}(f_F - j)$. For $B \ge 1$ the sum $\sum_v P_v^2 + P_F^2$ is zero exactly when all budgets hold and an accepting vertex is reached: when the budgets hold, $f_F \le B$, so the second product asks for exactly $f_F \ge 1$. Lemma 3 turns the truncated conditions into the original ones.

B.7 The main theorem

Theorem 2 (Main theorem). *For unary k, $\textsc{stcon}_{\mathrm{sf}} \in \mathsf{ReachUL} \cap \mathsf{LOGDCFL} \cap \mathsf{C_{=}L} \cap \mathsf{SC}^2$, and $\textsc{stcon}_{\mathrm{sf}} \in \mathsf{DSPACE}\big(O(\log^2 M/\log\log M)\big)$.*

Proof of Theorem 2. ReachUL: Theorem 5 or Theorem B.14; ReachUL = ReachFewL by Garvin et al. (2014). LOGDCFL and SC^2: Theorem 4, with Cook's simultaneous simulation giving one algorithm with both bounds. C$_=$L: Theorem B.15. DSPACE$(O(\log^2 M/\log\log M))$: Theorem 5. □

C Further results: budgets, completeness, thresholds and structure

C.1 Statements

Theorem C.1 (Total budget validation). *If the total numerical budget is polynomially bounded in the input length, then* RootBudget, PairBudget $\in \mathsf{ReachUL} \cap \mathsf{LOGDCFL} \cap \mathsf{C_{=}L} \cap \mathsf{SC}^2$. *Theorem 2 is the special case $\kappa(u,v) = k$.*

Theorem C.2 (Fail-closed compiler). *Let M_0 be a polynomial-time nondeterministic logspace machine, $\pi_x(c)$ the number of its computation paths on input x from the initial configuration to configuration c, and $\kappa_x(c) \ge 0$ logspace computable, with the total $\sum_c \kappa_x(c)$ bounded by a fixed polynomial. Then $L_{M_0,\kappa} = \{x : \pi_x(c) \le \kappa_x(c)$ for every c, and M_0 accepts $x\}$ is in* ReachUL $\cap$ LOGDCFL $\cap$ C$_=$L $\cap$ SC^2.

In words: a nondeterministic logspace computation that is *expected* to have few computation paths, with no promise and no restriction on the machine, can be replaced by a reach-unambiguous one that checks the expectation on every input and rejects where it fails.

Theorem C.3 (Completeness). *For every fixed $k \ge 1$, $\textsc{RootFew}_k = \{\langle G,s,t\rangle : N_G(s,v) \le k\,\forall v,\ N_G(s,t) \ge 1\}$ is* ReachUL*-complete under logspace reductions, also on DAGs; rooted k-few recognition is* ReachUL*-complete, and the two-cone language $\textsc{FBFew}_{k,1}$, which bounds walks out of s by k and into t by 1, is* ReachUL*-complete.*

Theorem C.4 (Threshold phase transition).
(i) With polynomially bounded thresholds, RootBudget *and* PairBudget *are in* NL*, and an* FL^{NL} *transducer outputs the capped counts $\min\{N^{\le B}(r,v), K+1\}$. More generally, for every #*L *function f and polynomially bounded logspace threshold τ, $\{x : f(x) \ge \tau(x)\} \in$* NL.
(ii) With the threshold in binary, the language is PL*-complete, even on DAGs on which s reaches t by construction.*
(iii) For a single pair and a fixed cap k, deciding whether a DAG has $N_G(s,t) \le k$ is NL*-complete.*

The complexity of validating few paths thus moves from inside ReachUL $\cap$ LOGDCFL to PL-complete exactly when the numerical threshold may become exponential, not when its notation changes (Table 8).

Theorem C.5 (The StrongFewL boundary). $\textsc{stcon}_{\mathrm{sf}}$ *is* StrongFewL*-hard under deterministic logspace many-one reductions. A proof that $\textsc{stcon}_{\mathrm{sf}} \in$* StrongFewL *would make it a natural complete problem for* StrongFewL.

The branch kernel (Theorem 6) and the block product (Theorem 7) are stated in Section 2, the remaining structure theorems are these.

Theorem C.6 (Finite forbidden-subdivision basis). *For every fixed $k \ge 1$ there is a finite family $\mathcal{F}_k$ of acyclic two-terminal multigraphs such that a finite directed graph is k-strongly few iff it has no directed cycle and contains no directed subdivision of a member of $\mathcal{F}_k$. Every member has between $k+1$ and $2k$ terminal paths, at most $2k$ vertices and at most $3k-1$ edges, and $|\mathcal{F}_k| \le 2^{O(k\log k)}$.*

Theorem C.7 (Bounded fixed point and mangrove normal form). *(i) G is k-strongly few iff $C = I + AC$ has a solution $C \in \{0,\dots,k\}^{n\times n}$ (A the adjacency matrix), the solution is then unique and equals the path-count matrix $(I-A)^{-1}$. (ii) For $k \ge 2$ the path-count matrix of a k-strongly-few graph factors over the nonnegative integers as $C = M_1 \cdots M_r$ with fewer than $4\log_2 k$ factors, each the path-count matrix of a mangrove, by a recursion of depth at most $1 + \lfloor \log_2 \log_2 k \rfloor$. For path-convex mangrove decompositions this is optimal: a chain of r diamonds needs at least $r+1$ blocks and depth at least $\lceil \log_2(r+1) \rceil$.*

Theorem C.8 (Density and width). *A simple k-strongly-few graph on n vertices has at most $\min\{\binom{n}{2}, n^2/4 + 3(k-1)n/4\}$ edges, and for every fixed $k \geq 2$ the maximum is $n^2/4 + \Theta_k(n)$. Every reachability interval $I(u,v) = \{x : u \rightsquigarrow x \rightsquigarrow v\}$ has width at most $N_G(u,v) \leq k$ and is the union of at most k directed u–v paths.*

Theorem C.9 (Tractable regimes). *Let $\lambda(G)$ be the smaller of the numbers of sources and sinks, and $b(G)$ the largest cyclomatic number of a biconnected block of the underlying undirected graph. Total strongly-few reachability is decidable in polynomial time and $O((k\lambda(G)+1)\log M)$ space, and in $2^{O(b(G))}M^{O(1)}$ time and $O(b(G)+\log M)$ space. In particular it is in L for fixed k and a fixed number of sources or sinks, in SC^d when $k\lambda(G) = O(\log^{d-1} M)$, and in L when $b(G) = O(\log M)$, as for chains of diamonds and oriented cacti.*

Read together, these results say that between two vertices of a few-path graph almost everything is forced, the rest factors into independent blocks, every violation contains one of finitely many small witnesses, and the whole path structure is a short product of mangroves.

Table 7 collects where each result lands, and Table 8 how the validation tasks are classified.

Table 7 | The classical placement of total few-path reachability. Every membership is proved on arbitrary inputs, with no promise.

Class	Result	Construction
ReachUL = ReachFewL	membership	certified finite-field lift (Theorem 5; oracle-free in Supplementary Section B.5)
LOGDCFL	membership	capped deterministic auxiliary pushdown (Theorem 4)
SC^2	membership	Cook's simulation of the same pushdown
$\mathsf{DSPACE}(\log^2 M/\log\log M)$	membership	the ReachUL machine, simulated as by Allender and Lange
$\mathsf{C}_=\mathsf{L}$	membership	one GapL zero test of squared vanishing products (Theorem B.15)
NL	membership	collision-averaged fingerprints (Theorem C.4)
StrongFewL	hardness; membership open	configuration graphs (Theorem C.5)

Table 8 | Validating few paths is easy until the threshold becomes exponential. Complexity of the validation tasks studied in Supplementary Section C.

Validation task	Classification
one pair, fixed cap	NL-complete
one rooted row, fixed k	ReachUL-complete (also on DAGs)
one rooted row or all pairs, polynomial total budget	in ReachUL ∩ LOGDCFL ∩ $\mathsf{C}_=\mathsf{L}$ ∩ SC^2
all pairs, unary k ($\textsc{stcon}_{\mathrm{sf}}$)	in ReachUL ∩ LOGDCFL ∩ $\mathsf{C}_=\mathsf{L}$ ∩ SC^2; StrongFewL-hard
all pairs, binary threshold	PL-complete

C.2 Proofs: budgets, compiler and completeness

Theorem C.10 (Root budgets in LOGDCFL). *If states, transitions and budgets are logspace enumerable and the total budget B is polynomially bounded,* $\textsc{RootBudget} \in$ LOGDCFL.

Proof. Reject if $B = 0$. Otherwise traverse the complete, untruncated walk-unfolding from r depth first, with suspended frames (a state and the index of its next transition) on the pushdown, and reject on entering node $B+1$. If the unfolding has more than B nodes, node $B+1$ is reached after polynomially many entries, even along an infinite branch. If the traversal completes, the unfolding is finite with at most B nodes; rerun it once per state q, counting nodes ending at q and rejecting at $\kappa(q)+1$, these recounts are exact. Accept iff some accepting state is reached. Invariant: either the complete unfolding has at most B nodes, or node $B+1$ is reached and the input is rejected. □

Proof of Theorem C.1. LOGDCFL and SC^2: Theorem C.10 with Lemma B.1 for pairs. ReachUL: the controller of Theorem 5. $\mathsf{C}_=\mathsf{L}$: the budget versions in Supplementary Section B.6. □

Proof of Theorem C.2. Generate the clocked configuration graph of M_0 on x in logspace, take the initial configuration as root and the accepting configurations as F, and attach the budgets κ_x. The graph has polynomial size, a computation path is exactly a rooted path, and the clock makes the graph finite even on rejected inputs, apply Theorem C.1. The compiler bounds computation prefixes from the initial configuration; it does not produce a machine with few paths between arbitrary configurations, which is the StrongFewL question. □

Proof of Theorem C.3. Membership follows from Theorem C.1 (for $\textsc{FBFew}$ apply the root budget to s in G and to t in the reversed graph). $\textsc{RootFew}_k$: for $k = 1$ it is $\textsc{stcon}_{\mathrm{ru}}$ (Lange, 1997). For $k > 1$ add a root r and k internally disjoint two-edge paths $r \to a_i \to s$, then $N_H(r,v) = k\,N_G(s,v)$ for old vertices, so

a bound of one becomes a bound of k. *DAGs.* Given a rooted threshold-one instance on n vertices, build layers $0..2n-1$ reproducing every edge between consecutive layers, and for every vertex v a fresh sink c_v with an edge from every layered copy of v; use $(s,0)$ as root and c_t as target. If the reachable part is acyclic but ambiguous, two simple rooted paths of length $\le n-1$ reach one sink; if a cycle is reachable, a simple prefix to a cycle vertex and the same prefix plus one tour give two walks of length $\le 2n-1$ to one sink. The k-fold prefix gadget gives every fixed k. *Recognition and two cones.* Rooted reach-unambiguity recognition is ReachUL-complete (Dinesh & Sasidharan, 2026), and the k-fold prefix reduces it to rooted k-few recognition. For FBFew$_{k,1}$, given $\langle G,s\rangle$ add r,z with edges $r\to s$, $r\to z$, and use r,z as source and target. □

C.3 Proofs: the threshold phase transition and the StrongFewL boundary

(i) Polynomial thresholds: collision-averaged fingerprints. Fix a root, budgets of total mass B and maximum $K\ge 1$, edge occurrences $e_1..e_m$, $q_0=(B-1)\binom{K+1}{2}+1$, a prime $p>\max\{m,q_0\}$ and a set $A\subseteq\mathbb{F}_p$ of q_0 points. For a walk P of length $d\le B$ let $\phi_\alpha(P)=(d,h_\alpha(P))$; for an endpoint v let W_v be the rooted walks of length $\le B$ ending at v, $c_v=|W_v|$ and $D_v=\sum_{\alpha\in A}|\phi_\alpha(W_v)|$.

Lemma C.11 (Collision-averaged decoding). *For every v and $t\le K$: $c_v\le t$ iff $D_v\le q_0 t$. Hence* $\min\{c_v,K+1\}=\min\{\lceil D_v/q_0\rceil,K+1\}$.

Proof. If $c_v\le t$ each point yields at most c_v images. Conversely pick $t+1$ distinct walks, two of the same length collide at most at $B-1$ points, and at a fixed α the number of lost images is at most the number of colliding pairs. Summing over A, the selected walks have at least $q_0(t+1)-(B-1)\binom{t+1}{2}>q_0 t$ images. □

Under a super-root over the lifts H_α, $\alpha\in A$, D_v is the number of reachable states in the fiber of v, which a logspace machine with an NL oracle counts; $\mathsf{L}^{\mathsf{NL}}=\mathsf{NL}$ because $\mathsf{NL}=\mathsf{coNL}$. Applied to the accepting computations of an NL machine, the same decoding gives $\{x: f(x)\ge\tau(x)\}\in\mathsf{NL}$.

(ii) Binary thresholds are PL-complete. *Membership.* Reject cyclic graphs ($\mathsf{NL}\subseteq\mathsf{PL}$); on a DAG each $N_G(u,v)$ is #L, a logspace machine has exactly $K+1$ accepting branches by guessing a number in $[0,K]$ bit by bit, so $(K+1)-N_G(u,v)\in\mathsf{GapL}$, and PL's closure under Boolean operations and NC^1 reductions (Beigel & Fu, 1997) handles the conjunction. *Hardness.* Take a probabilistic logspace machine with exactly p binary choices accepting on $A(x)$ of its 2^p paths. Between common endpoints a,b put a chain of $p+1$ diamonds ($C=2^{p+1}$ paths) in parallel with the layered configuration graph and set $K=C+2^{p-1}$. Every other pair has at most C paths and $N(a,b)=C+A(x)$, so the instance is accepted iff $A(x)\le 2^{p-1}$, this reduces the complement, and PL is closed under complement.

(iii) One pair, fixed cap. Reject non-DAGs in NL. On a DAG the complement guesses $k+1$ paths in parallel, storing their current vertices and pairwise-difference flags, so the language is in $\mathsf{coNL}=\mathsf{NL}$. Hardness: add a source with $k+1$ internally disjoint prefixes into the old source, the new count is 0 or at least $k+1$.

Proof of Theorem C.5. Let A be accepted by a StrongFewL machine with at most $q(|x|)$ paths between any two configurations and a unique accepting configuration. Its configuration graph G_x is logspace computable, acyclic and $q(|x|)$-strongly few, possibly after a clock or an accepting collector (a fixed polynomial factor); parallel transitions become private two-edge paths. Hence $x\in A$ iff $\langle G_x,C_0,C_{\mathrm{acc}},1^{q'(|x|)}\rangle\in\textsc{stcon}_{\mathrm{sf}}$. Membership is open because our reach-unambiguous machines control only the paths from their initial configuration; configurations unreachable from it may still be joined by many paths, and the oracle composition gives no all-pairs bound. □

C.4 Proofs of the structure theorems

Proof of Theorem 6. *Lower bound.* The unit s–t flows of H form a polytope of dimension $|E|-|V|+1=c(H)$ whose vertices are the indicator vectors of the paths (the constraint matrix is totally unimodular), a c-dimensional polytope has at least $c+1$ vertices. *Upper bound.* Fix a path P_0; $P\mapsto\chi_P\oplus\chi_{P_0}$ maps paths injectively into the cycle space over GF(2), of dimension $c(H)$. *Kernel.* In a trimmed H every vertex $x\ne t$ has out-degree at least one, and $\sum_{x\ne t}(d^+(x)-1)=|E|-(|V|-1)=c(H)$, so at most $c(H)$ vertices have out-degree at least two, and likewise for in-degree. Every surviving vertex is a terminal or one of these, and contracting a vertex of in- and out-degree one preserves paths and c, this gives at most $2c+2\le 2r$ vertices and $c+(2c+2)-1=3c+1$ edges. □

Proof of Theorem 7. A simple s–t path follows the unique route of blocks in the block–cut tree; it cannot enter an off-route block and return without repeating a cut vertex. Inside route block B_i it traverses one a_i–b_i path, and arbitrary choices concatenate into distinct s–t paths. Cyclomatic numbers add because blocks share only cut vertices. If $r\le k$, each block with more than one path contributes a factor of at least two. □

Proof of Theorem C.6. Let H be acyclic, trimmed and edge-minimal subject to $N_H(s,t)>k$, with mandatory terminal chains contracted, so $d^+(s)\ge 2$ and $d^-(t)\ge 2$; write $r=N_H(s,t)$, $c=c(H)$. Deleting an out-edge e of s leaves at most k paths, so $r_e\ge r-k$, the at least two out-edges partition the paths, so $r\ge 2(r-k)$, i.e. $r\le 2k$. In the unit-flow polytope the face $\{x_e=0\}$ of an irredundant inequality is a facet of dimension $c-1$ with at most k vertices, and a $(c-1)$-dimensional polytope has at least c vertices, so

$c \le k$. The excess-degree identities then give a core of at most $2c \le 2k$ vertices and $(2c-1)+c \le 3k-1$ edges. Any acyclic violation contains an edge-minimal violating subgraph, which is a subdivision of such a core; conversely subdividing preserves path counts. Finitely many multigraphs meet these bounds. □

Proof of Theorem C.7. (i) If G is k-strongly few it is acyclic, $A^n = 0$, and $C = I + A + \cdots + A^{n-1} = (I-A)^{-1}$ solves $C = I + AC$ with the path counts as entries. Conversely let $C \ge 0$ solve it and $\mathbf{r} = C\mathbf{1}$, then $\mathbf{r} = \mathbf{1} + A\mathbf{r}$, so $r_u > r_v$ for every edge $u \to v$ and a cycle would give a strict chain returning to its start; so G is acyclic and $C = (I-A)^{-1}$. (ii) For $1 \le \tau < k$ let $\mu(v) = \max_a N(a,v)$, $L_\tau = \{v : \mu(v) \le \tau\}$ and H_τ the rest. Since $\mu(v) \ge \mu(u)N(u,v)$, no path goes from H_τ to L_τ, $G[L_\tau]$ is τ-strongly few and $G[H_\tau]$ is $\lfloor k/(\tau+1) \rfloor$-strongly few. With $\tau = \lfloor \sqrt{k} \rfloor$ both parts are $\lfloor \sqrt{k} \rfloor$-strongly few, and ordering L before H, $C = \left(\begin{smallmatrix} C_L & 0 \\ 0 & I \end{smallmatrix}\right)\left(\begin{smallmatrix} I & B \\ 0 & I \end{smallmatrix}\right)\left(\begin{smallmatrix} I & 0 \\ 0 & C_H \end{smallmatrix}\right)$, the middle factor being a mangrove. The number of leaves satisfies $\ell(k) \le 2\ell(\lfloor \sqrt{k} \rfloor)$, so $\ell(k) \le 2\log_2 k$ and the number of factors is $2\ell - 1 < 4\log_2 k$, with depth at most $1 + \lfloor \log_2 \log_2 k \rfloor$. *Optimality.* In a chain of r diamonds with merge vertices $z_0..z_r$, two consecutive merge vertices cannot share a path-convex mangrove block, and two non-consecutive ones would force all intermediate ones into it; so the z_i lie in $r+1$ distinct blocks, and a binary recursion has depth at least $\lceil \log_2(r+1) \rceil$. □

Proof of Theorem C.8. *Density.* Every triangle of the underlying graph is transitively oriented, assign it to its long edge. An edge in r_e triangles has $1 + r_e$ paths between its endpoints, so $r_e \le k-1$ and $T \le (k-1)m$. Summing $|N(x) \cap N(y)| \ge d_x + d_y - n$ over edges, $3T \ge \sum_x d_x^2 - mn \ge 4m^2/n - mn$, which rearranges to the bound. For the lower bound take $|B| = k-1$, split the other $n-k+1$ vertices evenly into A and C, and add all edges $A \to B$, $B \to C$, $A \to C$. *Width.* For an antichain $X \subseteq I(u,v)$, concatenate a path $u \rightsquigarrow x \rightsquigarrow v$ for each x, two elements on one u–v path would be comparable, so $|X| \le N(u,v)$. □

Proof of Theorem C.9. *Terminal width.* For a root a of a DAG the unfolding tree has

$$L_a = 1 + \sum_{v:\, d^+(v) > 0} N(a,v)\,\big(d^+(v) - 1\big)$$

leaves, each at a sink, so $L_a \le k\,\tau^+(G)$ on a valid input, where $\tau^+(G)$ is the number of sinks, a depth-first traversal storing frames only at branching nodes keeps at most $L_a - 1$ frames. Choose the orientation with fewer sinks (reversal transposes the path-count matrix) and run the two passes of Theorem 4 with choice frames only, keeping the node cap $nk+1$, a length cap n and a stack cap $k\lambda(G) - 1$; on an invalid input any cap may reject, and if none does, the absence of a length-n walk proves acyclicity. *Block-cyclomatic width.* In a component with cyclomatic number c, enumerate the 2^c cycle-space vectors relative to a spanning tree, degree conditions and undirected connectivity characterize the directed simple s–t paths and detect directed cycles blockwise. On a DAG, follow the route of blocks (navigated with Reingold's algorithm (Reingold, 2008)), count paths per route block with at most b bits, and multiply with a counter saturated at $k+1$ (Theorem 7). □

D The mechanisms in detail

Table 9 lists what each mechanism rests on, the subsections state and prove each guarantee, and Supplementary Section E describes the implementation.

Table 9 | How the theory is used. For each mechanism, the result it rests on and the property it guarantees.

Mechanism	Result used	Guarantee
1. One-sweep table	Theorem 8(2), Theorem D.4	exact table in depth-of-trie gathers, or a report that the budget is exceeded
2. Forced spans and canonical split	Theorems D.7, D.10 and 6	same output; fewest model runs of any non-speculative decoder; committed tokens are the tokenizer's split
3. Walk kernel	Construction 2, Theorem D.4(ii)	the table's row recomputed each step; memory is the automaton
4. GPU stack	Theorems 4 and D.12	exact up to nesting depth 64, table indexed by the top two stack symbols
5. Certified blocks	Theorems D.15 and 7	token-identical to sequential greedy decoding
6. Budget rule	Lemma 3 and Theorem D.11	valid under every feasible limit; least restrictive such rule

D.1 Cost model, control machines and the step graph

Cost. Let $\tau(S)$ be the latency of one forward pass that appends S tokens to a cached prefix and returns logits at every new position. Single-stream decoding on a laptop is bound by memory bandwidth: on an M2 Pro, $\tau(4) = 1.36\,\tau(1)$, $\tau(8) = 1.58\,\tau(1)$ and $\tau(16) = 2.04\,\tau(1)$ on the 2B; on the 4B a pass costs 13 ms at $S = 1$, 21 ms at $S = 4$ and 39 ms at $S = 16$. Hence the cost of decoding is its number of passes. By causality, one pass over $v_1 \cdots v_S$ after a prefix u yields the same cache and logits as S single-token passes, up to floating-point reassociation.

Definition D.1 (Control machine). A control machine is $\mathcal{A} = (C, \delta, c_0, F)$ with configurations C, a partial step $\delta : C \times \Sigma \rightharpoonup C$ over bytes, start c_0 and accepting set F. δ^* extends δ to strings, $L(\mathcal{A}) = \{x : \delta^*(c_0, x) \in F\}$ and $\mathrm{live}(c) = \{b : \delta(c, b) \text{ defined}\}$. $\mathcal{A}$ is *trim* if an accepting configuration is reachable from every configuration reachable from c_0.

A DFA is the case of a finite $C = Q$, a deterministic pushdown is the case $C = Q \times R^*$ (Supplementary Section D.6). The vocabulary is a set $\mathcal{V} \subset \Sigma^+$ of byte strings plus a token EOS. Constrained decoding at c samples from the logits restricted to $\mathcal{A}(c) = \{t \in \mathcal{V} : \delta^*(c, t) \text{ defined}\} \cup \{\text{EOS} : c \in F\}$ (the mask), then moves to $\delta^*(c, t)$; if $\mathcal{A}$ is trim, every emitted prefix extends to a string of $L(\mathcal{A})$.

Definition D.2 (Step graph). For $c \in C$, P_c has vertices $(w, \delta^*(c, w))$ for the trie nodes w at which $\delta^*(c, w)$ is defined, edges $(w, c') \to (wb, \delta(c', b))$, and root (ε, c).

Proposition D.3. *For every deterministic control machine and every c, P_c is an out-tree, hence a mangrove. Each token $t \in \mathcal{A}(c) \setminus \{\text{EOS}\}$ corresponds to exactly one vertex of P_c, which carries $\delta^*(c, t)$.*

Proof. Every edge lengthens w, and a vertex (wb, c'') has the single parent $(w, \delta^*(c, w))$, because the trie fixes w and determinism fixes the configuration. □

D.2 The token table, with and without storing it

Theorem D.4 (Two ways to evaluate the step graph). *Let $\mathcal{A}$ be a DFA with states Q and $T[t, q] = \delta^*(q, t) \in Q \cup \{\bot\}$. (i)* One pass. *Walking the trie level by level with all $|Q|$ states as the columns of one array computes T in $\mathrm{depth}(T_\mathcal{V})$ gathers with total work at most $|T_\mathcal{V}| \cdot |Q|$, within a factor $|T_\mathcal{V}|/|\mathcal{V}|$ (2.4 for Qwen3.5) of the output size, and never more than evaluating every token from every state separately. (ii)* No table. *For any q, the row $T[\cdot, q]$ is computed by $|\mathcal{V}|$ independent walks from q, one per token, with $\sum_t |t|$ lookups and depth $\max_t |t|$, using only the automaton and the packed token bytes.*

Algorithm 5 All-state token table: all $|Q|$ states side by side, one trie level per step.

$S_0 \leftarrow [0, 1, \ldots, |Q| - 1]$ `// one row, the root; every state is a root`
for $d = 1, \ldots, \mathrm{depth}(T_\mathcal{V})$ **do**
  rows $\leftarrow$ nodes w at depth d whose parent row survived
  $S_d[w, \cdot] \leftarrow \delta(S_{d-1}[\mathrm{parent}_d(w), \cdot], \mathrm{byte}_d(w))$ `// one gather; ⊥ stays ⊥`
  $T[t, \cdot] \leftarrow S_d[\mathrm{node}(t), \cdot]$ for every token t ending at depth d
  drop the rows with $S_d[w, \cdot] \equiv \bot$
end for

Proof. (i) By induction on d, row w of S_d holds $(\delta^*(q, w))_{q \in Q}$, since $\delta^*(q, w) = \delta(\delta^*(q, \mathrm{parent}(w)), \mathrm{byte}(w))$ with $\bot$ absorbing, a row that is $\bot$ for every state has only $\bot$ descendants. Each level processes at most $|T^d_\mathcal{V}|$ rows of $|Q|$ entries. Mapping every non-root trie node w to a pair $(t, |w|)$ with w a prefix of t is injective, so $|T_\mathcal{V}| - 1 \le \sum_t |t|$. (ii) The walk of t from q is the root-to-t path of P_q (Proposition D.3) and uses no other token's walk. □

With (i) the table costs $3|Q||\mathcal{V}|$ bytes and is built in 0.2–0.6 s for our schemas; on the 104-state person schema the pass performs 61M lookups in 0.3 s and touches the trie essentially once, because free-text fields keep almost every trie node alive for some state. With (ii) a Metal kernel runs one thread per token each step (the walk kernel, Methods). It trades a stored table for recomputation along the single path of each token, the same trade the certified lift makes when it counts reachable states instead of storing walks.

D.3 Forced output costs no pass

Definition D.5 (Forced configurations and chains). c is *forced* if $c \notin F$ and $|\mathrm{live}(c)| = 1$, and a *decision point* otherwise. $\mathrm{chain}(c)$ is the byte string read by following the unique live byte from c while the configuration is forced. If $\mathcal{A}$ is trim, $\mathrm{chain}(c)$ is finite and ends at a decision point.

Definition D.6 (Segmentation policy; reference and forced-span decoders). A segmentation policy φ assigns to each configuration c a token string $\varphi(c) \in \mathcal{V}^*$ whose bytes are a prefix of $\mathrm{chain}(c)$, and to each sampling configuration a nonempty set $\mathcal{A}_\varphi(c) \subseteq \mathcal{A}(c)$; both depend on $\mathcal{A}$ and the tokenizer only. At a sampling configuration c, the reference decoder R_φ samples t from the logits restricted to $\mathcal{A}_\varphi(c)$, sets $c' = \delta^*(c, t)$, feeds the tokens of $\varphi(c')$ one pass each and continues at $\delta^*(c', \varphi(c'))$. The forced-span decoder F_φ is identical except that t and $\varphi(c')$ are fed in one pass.

Theorem D.7 (Pass optimality). *Let $\mathcal{A}$ be trim and φ a segmentation policy. (i) For every causal model, sampling rule and seed, F_φ and R_φ emit the same tokens and, in exact arithmetic, see the same logits at every sampling point. (ii) On an output with N sampled tokens, F_φ makes N passes and R_φ one per emitted token. (iii) If $|\mathcal{A}_\varphi(c)| \ge 2$ at every sampling configuration, every non-speculative decoder whose output distribution equals*

R_φ's for every model under greedy or softmax sampling makes at least N passes on every output with N sampled tokens.

Proof. (i) By induction over sampling points: both decoders see the same logits and draw the same t; R_φ feeds t and $\varphi(c')$ in $1 + |\varphi(c')|$ passes and F_φ in one, and by causality the caches and last-position logits coincide, the next configuration is the same because φ does not depend on the model. (ii) is counting. (iii) Fix a model p and an output y, with sampling positions $i_1 < \cdots < i_N$. *Step 1*: the decoder D evaluates $\ell(y_{<i_k})$ before it emits y_{i_k}; otherwise a model p' that differs from p only at $y_{<i_k}$, by changing the probability of y_{i_k} among the at least two admissible tokens, would leave D unchanged while R_φ's distribution changes. *Step 2*: these evaluations are in distinct passes, since a non-speculative D feeds y_{i_k} only in a pass after the one that produced $\ell(y_{<i_k})$, and $\ell(y_{<i_{k+1}})$ needs y_{i_k}. □

Corollary D.8 (Length independence). *Let φ commit the whole chain, let every accepting run from c pass through $c^\dagger$, let the runs from c to $c^\dagger$ form a trimmed two-terminal DAG H with r paths, and let no configuration of H other than $c^\dagger$ accept. Then F_φ samples at most $c(H) \le r - 1$ tokens that start in H before $c^\dagger$. A key costs no pass, and a field with r admissible values costs at most $r - 1$ passes, however long they are.*

Proof. Every sampled token starts at a decision point, which in H is a branch vertex (both live bytes lie on accepting runs through $c^\dagger$), and distinct sampled tokens start at distinct vertices; Theorem 6 bounds the branch vertices by $c(H) \le r - 1$. □

The DAG hypothesis matters: free-text fields create cycles, and Theorem 6 applies only to the acyclic parts between two cut configurations (keys, enum values, booleans, closing brackets).

D.4 Splitting forced text the way the tokenizer would

A byte-level BPE tokenizer is a pre-tokenizer π that cuts a string into blocks, followed by a per-block encoder: $\mathrm{tok}(s) = \mathrm{bpe}(s_1) \cdots \mathrm{bpe}(s_m)$ for $\pi(s) = (s_1, \ldots, s_m)$. Committing the canonical split of forced bytes w too early can contradict the canonical split of wx once the next bytes x arrive. Block boundaries are cut vertices of the segmentations the tokenizer can produce (Theorem 7), so only the last block is at risk.

Assumption D.9 (Locality of the pre-tokenizer). For a string s let $m(s)$ be the start of the last block of $\pi(s)$ or of the maximal trailing whitespace run of s, whichever is smaller. For every s and character a, every block of $\pi(s)$ that ends at or before $m(s)$ is a block of $\pi(sa)$.

Regex pre-tokenizers look ahead one character at whitespace (`\s+(?!\S)`), which is why the whitespace clause is needed. On the Qwen3.5 pre-tokenizer we tested the assumption on 720,000 random (string, character) pairs and 400,000 multi-character extensions and found no violation.

Theorem D.10 (Canonical commitment). *Let π satisfy Assumption D.9 and let o be the output so far, in configuration c. (i) For every x, every block of $\pi(o)$ ending at or before $m(o)$ is a block of $\pi(ox)$, so its canonical tokens can be committed now. (ii) If o does not end in whitespace and $\pi(o)$ is a prefix of $\pi(ob)$ for every $b \in \mathrm{live}(c)$, then $\mathrm{tok}(o)$ is a prefix of $\mathrm{tok}(ox)$ for every x with $\delta^*(c, x)$ defined.*

Proof. Let $e(s)$ be the largest block boundary of $\pi(s)$ that is at most $m(s)$. The blocks tiling $[0, e(s))$ are blocks of $\pi(sa)$ by Assumption D.9, and $e(sa) \ge e(s)$: the last block of $\pi(sa)$ contains position $|s|$ and so starts at or after $e(s)$, and the trailing whitespace run of sa starts at or after $m(s)$. (i) follows by induction on $|x|$. (ii) Write $x = bx'$ with $b \in \mathrm{live}(c)$; by hypothesis the last block of $\pi(ob)$ is $[|o|, |o| + 1)$, so $m(ob) = |o|$, every block of $\pi(o)$ ends at or before $m(ob)$, and (i) applied to ob and x' concludes. □

The implementation tests every ASCII byte and one representative per class of non-ASCII lead bytes; "$\pi(ob)$ has a boundary at $|o|$" was equivalent to the prefix condition in all 1.44M tested pairs. The whitespace clause is necessary: appending `\n` to a string ending in `\n␣` merges its last two blocks.

D.5 Valid outputs under a token budget

Count every emitted token against a budget R, an output that exhausts the budget ends there and is valid iff it ends in F. For a sampling configuration c let

$$\mathrm{need}(c) = \begin{cases} 0 & c \in F, \\ \min_{t \in \mathcal{A}_\varphi(c) \setminus \{\textsc{eos}\}} \mathrm{cost}(c, t) & \text{otherwise,} \end{cases} \qquad \mathrm{cost}(c, t) = 1 + |\varphi(c')| + \mathrm{need}(c''),$$

with $c' = \delta^*(c, t)$ and $c'' = \delta^*(c', \varphi(c'))$. With r tokens left the budget mask is $M_r(c) = \{t \in \mathcal{A}_\varphi(c) : \mathrm{cost}(c, t) \le r\} \cup \{\textsc{eos} : c \in F\}$.

Theorem D.11 (Least restrictive valid mask). *Let $\mathcal{A}$ be a trim DFA. (i) need is a shortest-path distance, computed by Bellman–Ford in at most $|Q|$ vectorized rounds. (ii) If $\mathrm{need}(c_1) \le R$, every output decoded under M_r, by any model and sampling rule, is in $L(\mathcal{A})$, whether it ends by EOS or by the budget. (iii) $t \in M_r(c)$ iff some continuation after t reaches F within the remaining budget, any mask that admits a token outside $M_r(c)$ admits a model whose output is invalid.*

Proof. (i) need satisfies the Bellman equation of a shortest-path problem with positive integer weights on the $|Q|$ sampling configurations. (ii) Invariant: $\mathrm{need}(c) \le r$ at every sampling point. If $c \notin F$, some t has $\mathrm{cost}(c,t) = \mathrm{need}(c) \le r$; for any admitted t the forced span fits and $r'' = r - 1 - |\varphi(c')| \ge \mathrm{need}(c'')$. If r reaches 0 at a sampling point, $\mathrm{need}(c) = 0$, i.e. $c \in F$. (iii) Following minimizing tokens realizes a continuation of weight $\mathrm{cost}(c,t)$, and every continuation has at least this weight, a model that greedily prefers a token with $\mathrm{cost} > r$ is cut outside F. □

The mask is inactive until the last $\max_{c,t} \mathrm{cost}(c,t)$ tokens (17–38 on our schemas), so outputs with room are unchanged; if $\mathrm{need}(c_1) > R$ no valid output fits, which the decoder can report before it starts.

D.6 Recursion: a deterministic pushdown with a capped stack

Definitions on a cycle of the schema's reference graph become nonterminals $N_1, \dots, N_C$; every other reference is inlined. Each N_j is a DFA over $\Sigma \cup \{\mathrm{call}_1, \dots, \mathrm{call}_C\}$, a call edge $q \xrightarrow{\mathrm{call}_j} r$ recording its return state r. A configuration is (q, σ) with $\sigma \in R^*$. On byte b the machine (1) moves if $\delta(q,b)$ is defined; else (2) if q has a call edge to N_j with $b \in \mathrm{first}(N_j)$, pushes r and moves to $\delta(\mathrm{start}_j, b)$; else (3) if q accepts and $\sigma = \sigma' r$, pops and retries b at r (a lazy return), and (4) after a move, while the state accepts and has no successor, pops and continues at the popped state (an eager return). The compiler checks $\mathrm{first}(N_j) \cap \mathrm{live}(q) = \emptyset$ at every call site and $\mathrm{live}(q) \cap \mathrm{follow}(N_j) = \emptyset$ at accepting states of called N_j with successors (an LL(1) condition), refuses grammars that violate them, and requires return states not to accept. A pop is *real* if it removes a symbol that was on the stack before the token began, and *exposed* if another move of the token's run follows it.

Theorem D.12 (Stack locality). *For a token t and a state q, the run of t from (q, σ) depends on σ only through the symbols revealed by its exposed real pops: there is a decision tree of depth $e \le |t|$, branching only on those symbols, whose leaves give the final state (or "continue at the popped state"), the number of real pops and the pushed string. If $e = 0$, one table entry serves every stack.*

Proof. Simulate the run with the initial stack unknown, tracking the symbols t pushes. While the top symbol was pushed by t it is known. A pop below these symbols is real; if no move follows it, its symbol enters the result only as "continue at the popped state", and otherwise the run branches on its value. Each exposed real pop consumes one original symbol and is followed by at least one byte or pop of t, so the depth is at most $|t|$. □

On Qwen3.5's vocabulary and our recursive schemas only 7–12 tokens at 4–11 states are context-dependent, and $e \le 2$ suffices except for the token }}}, which is masked, as are tokens that would push more than three symbols; masking them removes no byte string from the language, because every single byte is a token. The depth cap changes the language to $L(\mathcal{A}) \cap \{\text{nesting depth} \le 64\}$, and a configuration left with no admissible token stops with an explicit error.

Proposition D.13 (Sound budgets for pushdowns). *Let $b_n(q)$ be the length of the shortest byte string that completes the current nonterminal from q: the least fixpoint of $b_n(q) = 0$ for accepting q and otherwise $\min\{1 + b_n(\delta(q,b)),\ b_n(\mathrm{start}_j) + b_n(r)\}$ over bytes and call edges. Then $\mathrm{need}^+(q,\sigma) = b_n(q) + \sum_{r \in \sigma} b_n(r)$ is the length of the shortest byte completion of (q,σ), and if every single byte is a token, admitting a token only if $1 + \mathrm{need}^+(c') \le r$ guarantees validity in the sense of Theorem D.11(ii).*

Proof. Every completion decomposes into a completion of the current nonterminal, the return, a completion of the return state's nonterminal, and so on, the LL(1) conditions place the pops exactly there, so need^+ is exact. Every token and forced span covers at least one byte and forced bytes lie on every completion, so the invariant $\mathrm{need}^+ \le r$ is preserved, the single-byte token that begins a shortest completion is always admitted. □

The bound is sound but not least restrictive, because a token may cover several bytes.

D.7 Independent parts in parallel, certified

Lemma D.14 (Concatenation at cut states). *Let $\mathcal{A}$ be a trim DFA and $c_1, \dots, c_m$ states such that every accepting run from c_0 visits them in order, each once. Then every $w \in L(\mathcal{A})$ factors uniquely as $x_0 x_1 \cdots x_m$ with $\delta^*(c_i, x_i) = c_{i+1}$ ($c_{m+1} \in F$), and every such concatenation is in $L(\mathcal{A})$.*

Proof. The run of w is unique and visits the c_i in order once each; cut it there. Conversely the runs on the x_i concatenate to an accepting run. □

For an object schema whose keys are all required, every key is a forced chain, so the state after each key is a cut state and the language is a product of one value language per field: the automaton analogue of Theorem 7. The model's distribution over the values is not a product, so the decoder uses the factorization only to draft, and certifies the draft.

Theorem D.15 (Certified block decoding). *Fix φ and greedy decoding. Draft each field independently from the configuration at which the sequential decoder would begin it, assemble the draft D, replay it through $\mathcal{A}$ with φ to obtain the configuration q_i before every sampled position i; compute the logits at every position of D in one pass, and accept position i iff D_i is the masked argmax at q_i. In exact arithmetic the longest accepted prefix, extended by the argmax at the first rejected position, is a prefix of the sequential greedy output; repeating from that prefix produces exactly the sequential output, and each round adds at least one sampled token.*

Proof. Between sampled positions both the sequential output Y and D consist of the forced tokens φ prescribes. Let i^* be the first rejected sampled position. By induction over earlier sampled positions the replay's configuration and prefix equal the sequential decoder's, so the certification logits equal its logits (Theorem D.7(i)) and acceptance means $D_i = Y_i$. Hence $D_{<i^*} = Y_{<i^*}$ and the masked argmax at i^* is Y_{i^*}. □

This is speculative verification (Leviathan et al., 2023; Chen et al., 2023) with drafts that the grammar's factorization makes independent. In floating point the certification pass computes logits over many positions at once in fp16, so a near-tie could flip, all 36 measured outputs were token-identical. A round costs the longest draft plus one certification pass, so in the worst case (one new token per round) it is slower than sequential decoding, a cap of 24 rounds followed by sequential decoding bounds this.

D.8 The reply grammar for tool calls

A reply to a request that offers tools decodes under

$$\textsc{Reply} = \textsc{Free}\,\big(\textsc{Call}\,(\hookleftarrow \textsc{Call})^*\big)^?,$$
$$\textsc{Call} = \texttt{<tool_call>} \hookleftarrow \texttt{<function=}\textsc{Name}\texttt{>} \hookleftarrow \textsc{Param}^* \texttt{</function>} \hookleftarrow \texttt{</tool_call>},$$

with $\textsc{Param} = \texttt{<parameter=}P\texttt{>} \hookleftarrow \textsc{Val} \hookleftarrow \texttt{</parameter>} \hookleftarrow$, Name an alternation of the offered tool names, Val any text not containing `</parameter>`, and Free any text containing neither `<tool_call>` nor `<function=`.

Proposition D.16 (Exact "not containing" expression). *Let $\ell_1, \dots, \ell_m$ be strings that start with < and contain no other <, and ρ_i be ℓ_i without its first character. A string contains no ℓ_i iff no segment that follows a < (segments being the maximal <-free substrings) starts with some ρ_i. Hence*

$$\mathrm{NotContaining}(\ell_1, \dots, \ell_m) = \texttt{[^<]*}\Big(\texttt{<}\big(\textstyle\bigcup_p p\,\texttt{[^<}c_1 \cdots c_j\texttt{][^<]*} \cup \bigcup_p p\big)\Big)^*,$$

where p ranges over nodes of the trie of the ρ_i that are not complete and have no complete ρ_i as a prefix, and $c_1 \cdots c_j$ are the children of p.

The construction is tested against brute force on all strings up to length 6 and on 2,000 random strings per literal set.

E Implementation

E.1 Hardware, models and protocol

One Apple M2 Pro (19-core GPU, 16 GB unified memory, of which the GPU may wire 10.7 GB). Qwen3.5-2B (1.1 GB resident) and Qwen3.5-4B (2.26 GB) in 4-bit precision. The models are hybrid: three of every four layers are recurrent (Gated DeltaNet (Yang et al., 2025)) and keep a fixed-size state; on the 4B the KV cache costs 32 KB per token per stream and the recurrent state 50 MB per stream. Workloads are six JSON extractions with schemas (a person record, a tool call, a product, an event with attendees, a review classification and a contact card), four or five recursive schemas (trees of nodes), coding agents that call tools, and 40 LiveCodeBench problems (Jain et al., 2024). We report time to finish a task, model runs and answer quality. One GPU job runs at a time; configurations are interleaved inside one process and medians reported. Baselines run as controllers in our loop with our kernels when the comparison is about the control side, and in their own process, as shipped, when it is about the engine. Memory is read from MLX's own counters (active and peak GPU bytes), the resident set size is meaningless for unified memory.

E.2 Constraint compilation

A JSON schema is turned into a regular expression by outlines-core, a regex constraint is used as given. Our parser builds a Thompson NFA over bytes (character classes over code points are expanded into UTF-8 byte sequences), determinizes it by subset construction with a state budget (200,000 by default; beyond it compilation stops with an error), trims states from which no accepting state is reachable, and minimizes by Moore refinement. Lookaround and backreferences are rejected. Special tokens have no bytes and are never allowed by a constraint, added tokens that are not special (Qwen's `<tool_call>`, `</think>`) receive their surface bytes. The trie is stored level by level.

E.3 The walk kernel

The automaton is kept on the device as $256|Q|$ int16 entries ($\perp = -1$), with the accepting vector and an end-of-sequence vector over the vocabulary. The vocabulary is packed once: every token's bytes concatenated with offsets, 1.5 MB plus 1 MB of offsets for the 248k tokens, 2.9 MB with the rest. One Metal kernel (`mx.fast.metal_kernel`, grid $|\mathcal{V}| \times B$, threadgroup 256) runs per step: thread t of row b reads state q_b, walks its token's bytes until the end or $\perp$, and writes the state reached. The mask is "row $\neq \perp$, or q accepting and the token is EOS", combined with the boundary cover of Supplementary Section E.4, the next state is the row entry of the sampled token. States with at most 4,096 allowed tokens keep a sparse output head (allowed ids computed once on the CPU in about 4 ms, at most 16 KB per state). The batch engine builds one combined automaton table, with state 0 as the free state, and launches one kernel over all rows.

E.4 The forced planner

Given the previous sampled token y and the configuration q after it, the planner follows the unique live byte while the configuration is forced (the forced string w), and then: (1) cuts $s = \mathrm{bytes}(y)\, w$ to its longest valid UTF-8 prefix; (2) computes the pre-token block starts of s and holds back the last block, unless all of w survived, s does not end in whitespace, and for every byte allowed after w the pre-tokenizer puts a boundary at $|s|$ (the stable-tail test of Theorem D.10); (3) if the end of y is not a block boundary of s, re-encodes y together with w (healing; no cache rollback is needed in the synchronous loop); (4) tokenizes the committed segment with the model's tokenizer and checks that the tokens spell it, and (5) returns the tokens, the end configuration and a *boundary cover*: if the held-back bytes form one pre-token block that is a single vocabulary token of length ℓ, the next token must span at least ℓ bytes, so the model chooses how the block continues in one token, as the tokenizer would. Healing is off with the walk kernel, as it is in practice with the table (its threshold exceeds 200 tokens on our schemas).

E.5 The pushdown constraint

Definitions on a cycle of the schema's reference graph become nonterminals; each is a DFA with $256 + C$ columns, a call of N_i being a private code point mapped to column $256 + i$. The token tables are the level pass of Algorithm 5 run for all states at once with rows that carry a symbolic stack: pushes inside a token are recorded (at most three) and matched by pops inside it, a real pop splits the row into one row per candidate return state, at most two real pops are resolved and deeper ones masked. A token whose walk makes no real pop, or ends exactly at its single real pop, gets a context-free entry in the $|\mathcal{V}| \times |Q|$ table, the few others live in a context table indexed by the top two stack symbols. The device state vector is $[q, \mathrm{ptr}, \mathrm{stack}[0..63], S]$ with $S = \sum_{r \in \sigma} b_n(r)$ for budgets (Proposition D.13). For a tree of named nodes the root rule is `{"name":S,"children":[(call(,call)*)?]}`: 37 states, one call site and 6 context-dependent tokens at 4 states on Qwen's vocabulary.

E.6 Certified block-parallel decoding

Plan. The planner accepts object schemas whose properties are all required, flattening nested objects into leaves, arrays and free strings stay single blocks. Two dummy documents with different placeholder values locate, for every leaf, the state after its key and its exit state, which must agree between the two (the automaton is minimal). A block starts where the sequential decoder starts sampling, with the bare key path as its context. *Wave.* Every block is a row of the batch engine, started from the prompt's cache (array handles shared, recurrent state copied), with a first-wave budget of 24 steps, or $1.5\times$ the block's mean length plus 2 once the schema has been seen. *Certificate.* The samples are replayed through a fresh controller to obtain the draft D and the state before every sampled token; one pass over D gives the logits at every position, and the masked argmax is compared with D_i. Failing blocks continue from their first failing position with the correct token, for at most 24 rounds (never approached: at most 3 rounds on the 2B and 4 on the 4B). Rejected designs are listed in Supplementary Section J.

E.7 Rows on demand

A host-side variant keeps the automaton, the forced chains and a bounded dictionary of rows, each computed on first use by the level pass restricted to one root (about 4 ms per state on the CPU). Memory is proportional to the visited states: 17 rows (13 MB) instead of the 824 MB table for the typed tool grammar (Table 20). It runs on the synchronous engine only, the walk kernel supersedes it.

E.8 Engine: kernels, pipelining and batching

4-bit matrix–vector kernels for 1–16 rows dequantize weights straight into 8×8 simdgroup-matrix fragments, a two-level autotuner picks, per shape and then per pipelined model step, among these and MLX's kernels (first load on the 4B: about 19 minutes; later loads 3 s). One fused Metal kernel per layer performs the whole Gated DeltaNet decode step for up to 16 tokens. M1/M2 GPUs have no native bf16 arithmetic, so bf16 checkpoints run in fp16, which is also closer to float32 (Table 18). *Pipelining:* step $n + 1$ is enqueued with the sampled token and the next state as unevaluated device values while the GPU runs step n, so host graph construction (1.2 ms) and command encoding (1.8 ms) overlap the GPU's 7 ms; outputs are identical to the synchronous loop. *Batching:* streams join and leave a running batch at any step, every row feeds exactly one token per step, new prompts are prefilled in one right-padded batch, and a radix-trie prompt cache serves exact prefix hits. *Sampling:* greedy, or temperature with top-p and top-k under mlx-lm's rule.

E.9 Server, reply grammar and agents

The engine is packaged as an OpenAI-compatible server (the `chat/completions` endpoint with `tools` and `json_schema` response formats) and as a command that configures the OpenCode coding agent. Every request is a row of the running batch, and an assistant turn with tool calls is re-rendered verbatim, so it is served from the prompt cache (647 of 675 prompt tokens on a tool round trip). A reply to a request that offers tools decodes under a regular language: free text that cannot start a call, then zero or more calls, each a fixed tag structure around a tool name and parameters (Proposition D.16 gives the exact "not containing" expression). In the typed grammar every tool's parameter names and value types are fixed (1,106 states), which only the walk kernel can afford. Every tag is a forced span (20–30% of a call's tokens), and a reply cut by the token budget still closes its call.

E.10 Exactness and tests

In exact arithmetic none of the mechanisms changes what greedy decoding produces, except the canonical policy, which changes the segmentation of forced text on purpose. Measured: certified blocks are token-identical to sequential decoding on all 36 outputs, the walk kernel's rows equal the table's columns for every state, single-stream and batched; pipelined and synchronous loops agree, the pushdown tables equal byte-by-byte stepping on every reachable configuration of a small vocabulary with stacks to depth 4 and on brute force over all 248k tokens at sampled prefixes; budgets survive a simulated stopping rule with adversarial token choices. 234 automated tests (44 also against the 4B) cover the capped traversal and path counts against brute force on random graphs, the regex compiler against Python's `re`, the forced planner against the tokenizer, every kernel family against stock mlx-lm in both dtypes, the pushdown, the walk kernel, certified blocks, the server and sampling. Comparisons with other engines are subject to floating-point near-ties (Table 17).

F Speed: all measurements

Setup as in Methods. "Stock" means mlx-lm as shipped, in its own process, with its own kernels and bf16 arithmetic. Every number is tagged in the released result files with the script that produced it.

F.1 Speed of a single constrained stream

Table 10 | Three engines in one loop. Decode tokens/s on the same prompts with the same kernels, greedy, medians of three interleaved rounds. Six JSON schemas (regular) and four recursive schemas. Runs/token are forward passes per output token (2B); fidelity is the share of the unconstrained answer's values kept. For llguidance and XGrammar the better of the plain and the overlapped integration is shown.

		Tokens/s		Runs per	Fidelity		Output tokens	
Path	Engine	2B	4B	token	2B	4B	2B	4B
Regular	free decoding	137	66	1.01	—	—	570	597
	STCON	**173**	85	0.69	**0.94**	0.87	327	329
	llguidance, mask only	103	55	1.00	0.94	0.87	343	333
	llguidance, fast-forward	143	73	0.72	0.94	0.87	343	333
	XGrammar, mask only	103	56	1.02	0.92	**0.95**	343	340
	XGrammar, jump-forward	159	**86**	0.58	0.65	0.71	295	316
Recursive	free decoding	135	67	1.01	—	—	592	845
	STCON (pushdown)	161	84	0.71	—	—	469	382
	llguidance, mask only	103	56	1.00	—	—	310	418
	llguidance, fast-forward	141	73	0.72	—	—	310	418
	XGrammar, mask only	103	55	1.02	—	—	258	226
	XGrammar, jump-forward	**182**	**91**	0.52	—	—	334	155

Decode times for the six JSON extractions are 1.89 s (2B) and 3.89 s (4B) for STCON, against 2.41 s and 4.59 s for llguidance's fast-forward and 3.34 s and 6.08 s for its mask-only mode ($1.28\times$, $1.18\times$, $1.77\times$, $1.56\times$). The gain decomposes into fewer runs (0.69 against 1.00 per token) and cheaper runs (8.3 against 9.7 ms on the 2B, because the mask never leaves the GPU): $343 \times 9.7\,\text{ms} = 3.3\,\text{s}$ becomes $226 \times 8.3\,\text{ms} = 1.9\,\text{s}$. Each library's overlapped integration, in which its CPU mask is computed while the GPU runs the step, gains at most 2%: the cost is not the mask computation (3–13 μs for XGrammar, 60–270 μs for llguidance) but the CPU-to-GPU copy and the synchronous loop it forces.

On the recursive path the engines write different trees (469 against 310 output tokens on the 2B), so throughput, not decode time, is the comparable number. An independent run over six JSON and five recursive schemas (Table 11) gives the same picture. End to end on 18 extraction jobs on the 2B (prefill plus decode), STCON takes 8.0 s against 12.3 s for llguidance, and 6.8 s when the prompt carries only the keys and enum options, agreement with the unconstrained answer is 0.93 in every mode and all outputs are valid.

On the expression task every engine fails: the model nests without end; ours stops at the depth cap, llguidance at the token limit. Under the canonical policy one phrasing of the discussion task duplicates replies until the limit, the same policy yields the complete Solar-System tree (all five planets, 92 tokens) where llguidance stops after Earth (46 tokens). Single greedy samples of a 2B model are path-sensitive in both directions.

Table 11 | An independent run: six JSON schemas and five recursive schemas. Tokens per second count forced tokens; ms per sampled token is the derived time per model run.

	Qwen3.5-2B				Qwen3.5-4B			
Mode	tok/s	runs/tok	forced	ms	tok/s	runs/tok	forced	ms
free decoding	138	1.01	0%	7.2	63.2	1.01	0%	15.7
JSON, STCON	**180**	0.69	31%	8.0	**82.3**	0.69	31%	17.6
JSON, llguidance fast-forward	136	0.72	28%	10.2	68.6	0.73	27%	20.0
recursive, STCON	**170**	0.72	28%	8.2	**78.6**	0.70	30%	18.2
recursive, llguidance fast-forward	143	0.71	29%	9.9	67.4	0.72	28%	20.6
recursive, llguidance mask only	94	1.00	0%	10.6	—	—	—	—

Table 12 | Recursive schemas in detail. Five tasks (a Solar-System tree, an expression tree, a threaded discussion, a directory tree, a taxonomy) × three phrasings, Qwen3.5-2B, greedy, 400-token limit; ms per token over valid outputs only.

Mode	Valid	Model runs	Forced tokens	ms per token
STCON pushdown, canonical policy	11/15	1,452	921	**5.9**
STCON pushdown, commit-all	12/15	1,304	875	**5.8**
llguidance	12/15	2,204	0	9.4
llguidance, fast-forward	12/15	1,309	895	7.0

Typed tool calls through the server. Eight concurrent agent requests, two rounds. On the 2B the typed grammar with the walk kernel produces 46 of 46 calls with every argument present and of its declared type at 366–403 tokens/s (30% of tokens forced, since parameter names are chains), against 43 of 52 typed calls with the untyped grammar and token tables at 372–410 tokens/s (20% forced). On the 4B, 8 of 8 calls are typed either way (131 against 139 tokens/s): the typed grammar turns a habit into a guarantee at a cost of 6%.

F.2 Certified blocks

Table 13 | Certified block-parallel decoding produces the sequential tokens with fewer runs. Six schemas, three prompt variants each, greedy. On the 4B, block runs split into wave steps and certification passes.

	Qwen3.5-2B			Qwen3.5-4B			
Schema	Sequential	Blocks	Exact	Sequential	Blocks	(waves + cert.)	Exact
person	84	33	3/3	84	33	(30 + 3)	3/3
tool_call	35	17	3/3	35	20	(14 + 6)	3/3
product	112	53	3/3	112	46	(40 + 6)	3/3
event	169	96	3/3	167	109	(94 + 10)	3/3
classification	126	112	3/3	142	159	(148 + 7)	3/3
contact_card	146	63	3/3	146	117	(108 + 6)	3/3
all 18 outputs	672	**374**	18/18	686	**484**		18/18
rounds (max / median)		3 / 2			4 / 2		
decode wall time	5.9 s	7.4 s		12.2 s	18.6 s		

First-round certification accepts 438 of 654 sampled tokens on the 2B and 365 of 668 on the 4B (the larger model commits to earlier values more). On the M2 Pro a wave step over 5–9 rows costs 14–20 ms against 7 ms for one row, and a certification pass over 34–50 tokens costs 33–55 ms, which is why wall time does not yet drop.

G The whole engine: throughput, agents and LiveCodeBench

These comparisons are against stock mlx-lm, in its own process with its own kernels and bf16 arithmetic, so they include our batched kernels as well as the constraint mechanisms.

With the same prompt on all eight streams the numbers do not move (273 against 273 tokens/s on the 4B), so nothing is shared between streams. At 128 streams the recurrent states hold 2.3 GB and the wider matrix multiplications cost more than the extra rows return. Single-stream decode on the 2B falls only from 138 to 118 tokens/s as the context grows from 128 to 8,192 tokens, because only 6 of the 24 layers attend over the context; prefill runs at about 1,700 tokens/s. In batched constrained serving (18 JSON jobs, schema in the prompt) our engine is $1.33\times$ and $1.31\times$ faster than mlx-lm's batch generator with llguidance at 4 and 8 slots and level at 16 slots, where a step is bound by arithmetic; with the compact keys-only hint (118 instead of 212 prompt tokens) it is 1.5–$1.6\times$ faster at every slot count.

At eight agents our wall time is longer although throughput is $1.57\times$ higher, because our agents generated 80% more tokens (3,778 against 2,104): the engines do not make the same decisions, and this small model finishes about half the tasks under either. Throughput is the like-for-like number. Through the OpenAI-compatible server (`stcon bench -n 8 -rounds 3`, 2B), ours serves 1,928–1,954 output tokens per round in 4.6–4.8 s

Table 14 | Free decoding, total tokens/s by number of streams (256 tokens per stream, no constraint; plotted in Fig. 6). "mlx-lm on ours" is mlx-lm's batch generator on our patched model in the same process.

	Qwen3.5-2B				Qwen3.5-4B		
Streams	Stock	mlx-lm on ours	STCON	vs stock	Stock	STCON	vs stock
1	127	140	142	1.12×	61	64	1.04×
2	172	251	259	1.51×	82	116	1.41×
4	186	341	339	1.82×	87	160	1.83×
8	188	575	576	3.06×	87	271	3.13×
16	342	751	758	2.22×	157	341	2.17×
32	624	844	873	1.40×			
64	615	837	952	1.55×			
96	613	—	962	1.57×			
128	613	830	892	1.45×			

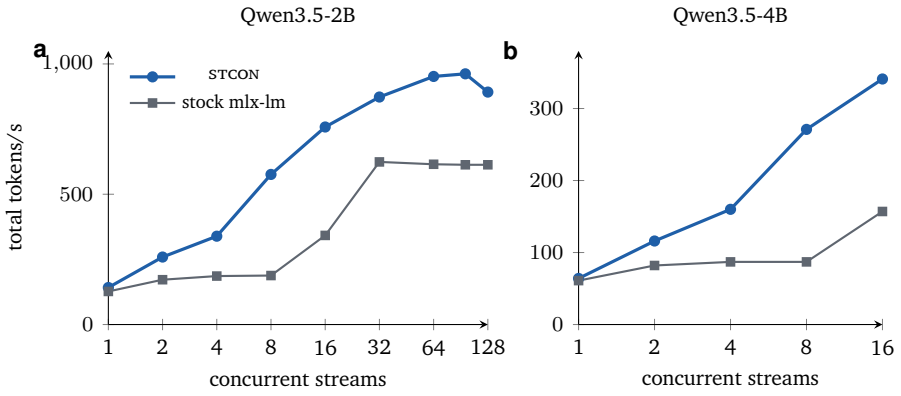


Fig. 6 | Stock mlx-lm stalls between four and eight streams; our kernels keep scaling. Free decoding, 256 tokens per stream, no constraint (Table 14). mlx-lm's own loop on our patched model reaches our numbers, so the gain is in the kernels and fp16 arithmetic, not the loop.

Table 15 | Parallel coding agents. Built-in suite of small coding tasks, Qwen3.5-2B, greedy, every reply constrained to one well-formed tool call. Stock is mlx-lm's batch generator with llguidance; both use continuous batching and the same prompt cache (94–96% hits).

	16 agents		8 agents	
	Stock	STCON	Stock	STCON
Wall time	74.6 s	**29.3 s** (2.55×)	**19.4 s**	22.1 s (0.87×)
Throughput	125.5 tok/s	**271.1 tok/s** (2.16×)	108.6 tok/s	**170.6 tok/s** (1.57×)
Agents finished	9/16	**12/16**	5/8	**6/8**
Checks passed	**11/16**	10/16	**5/8**	4/8

(median 410 tokens/s, 366–454 of the tokens forced) against 597–792 tokens in 4.5–7.3 s for stock (134 tokens/s); since stock replies are shorter, the fair reading is about three times the tokens in the same wall time, every reply parsing. Four OpenCode coder agents through one `opencode serve`: 19 s wall, prompt cache 61%, three of four tasks correct.

Table 16 | LiveCodeBench on Qwen3.5-4B. The 40 most recent easy and medium problems of release v6 (March–April 2025; 20 functional, 20 stdin), official prompt, a problem passed only when every public and hidden test passes. Tokens/s is per stream for one stream and total for batches. †Thinking on, 20 problems, 6 slots, temperature 0.6, top-p 0.95, top-k 20.

Cap	Engine	Pass@1	Decode tokens	Tokens/s	Wall time	Cut at cap
1,024, greedy	STCON, one stream	8/40	28.2k	63.4	481 s	23
	mlx-lm, one stream	11/40	27.6k	62.3	499 s	22
	STCON, batch 8	9/40	27.4k	**160.4**	**171 s**	22
	mlx-lm, batch 8	10/40	28.3k	74.5	379 s	23
4,096, greedy	STCON, one stream	9/40	96.0k	59.1	1,667 s	22
	mlx-lm, one stream	14/40	87.2k	61.1	1,482 s	17
	STCON, batch 8	12/40	93.3k	**196.1**	**476 s**	20
	mlx-lm, batch 8	13/40	93.0k	78.7	1,181 s	19
16,384, thinking†	STCON, batch 6	6/20	275k	119.8	2,296 s	15
	mlx-lm, batch 6	1/20	4.4k	—	out of memory	—

With thinking off, the 4B reasons in prose at length before its code; half the answers hit the 4,096-token cap, and the outputs of any two engines are identical on only 2–6 of 40 problems (mlx-lm's own single-stream and batched paths agree on 4 of 40). Pass rates therefore differ by a few problems in either direction between engines that compute the same model, and every divergence is a near-tie (Table 17). With thinking on and greedy decoding the 4B loops (16 of 20 problems run to the 16k cap, 11 repeating one sentence); with the model card's sampling settings 5 of 20 finish on our engine and all 5 pass, while mlx-lm's batch generator runs out of memory at six 16k-token streams after one finished answer.

Table 17 | Every divergence between engines is a near-tie, and float32 usually sides with our fp16 engine. For every problem where an engine's output parts from mlx-lm's single stream, the prompt plus the common prefix is run in float32 and the logit gap between the two competing tokens read.

			Logit gap		float32 prefers	
Cap	Engine	Divergent	median	max	mlx-lm	the other
4,096	STCON, one stream, fp16	35	0.056	0.52	3	**32**
	STCON, batch 8, fp16	34	0.062	0.52	2	**32**
	mlx-lm, batch 8	36	0.145	1.83	22	14
1,024	STCON, one stream, bf16	38	0.097	0.33	23	15

Table 18 | fp16 is closer to float32 than bf16. Six prompts, 96 teacher-forced positions each.

Model	Dtype	Top-1 agreement	Mean KL	p99 KL	Max KL	Max $\lvert\Delta\text{logit}\rvert$
4B	bfloat16 (mlx-lm)	0.9965	1.67×10^{-4}	2.0×10^{-3}	3.0×10^{-3}	5.60
4B	float16 (STCON)	**1.0000**	3.1×10^{-6}	4.6×10^{-5}	9.1×10^{-5}	0.31
2B	bfloat16	0.9965	2.6×10^{-4}	2.8×10^{-3}	5.1×10^{-3}	0.71
2B	float16	**1.0000**	3.6×10^{-6}	3.6×10^{-5}	5.3×10^{-5}	0.09

On the 2B, greedy generations of the two engines diverge only at near-ties (competing tokens within 0.08 logits in float32); on twelve generic 4B prompts, 5 of 12 200-token generations are identical in either dtype, all gaps within ± 0.27, float32 siding with ours on 6 of 7 in fp16.

H Memory: all measurements

Memory is read from MLX's own counters (active and peak bytes on the GPU) inside each engine's process, the resident set size is meaningless for unified memory.

What remains of compilation with the walk kernel is the automaton: 0.02–0.5 s for the schemas and 25–28 s for the tool grammar, whose subset construction, not its table, is the cost. On the 4B, a process that had already built the earlier grammars' tables next to the model no longer had room for the typed grammar's 824 MB table: it took 332 s to build and decoded at 6 tokens/s under swap pressure, while the walk kernel in the same process ran at 25 tokens/s, the rows above are from a fresh process.

With the walk kernel the automata and the packed vocabulary are about 50 MB of the 0.6 GB above the model on the 2B, the rest is MLX's buffer cache. Throughput with the walk kernel stays within the run-to-run spread (114–209 against 107–218 tokens/s on the 2B).

Table 19 | Constraint memory against grammar size: token table against walk kernel. Pipelined engine, greedy, no healing on either side, tokens/s median of three interleaved rounds. Walk memory includes the 2.9 MB shared vocabulary. Outputs are identical wherever the table exists.

		Token table		Walk kernel		tok/s, 2B		tok/s, 4B	
Grammar	States	memory	build	memory	per step	table	walk	table	walk
person	104	77 MB	0.7 s	2.9 MB	12–14 μs	156	153	75	75
tool_call	164	122 MB	1.0 s	2.9 MB	7–9 μs	263	256	124	121
product	192	143 MB	1.3 s	2.9 MB	8–9 μs	191	189	90	89
event	178	133 MB	1.3 s	2.9 MB	7–9 μs	168	161	80	78
classification	158	118 MB	1.0 s	2.9 MB	7–9 μs	159	169	82	83
contact_card	246	183 MB	1.7 s	3.0 MB	7–8 μs	183	180	88	87
48 fields	986	735 MB	9 s	3.3 MB	9–14 μs	274	267	131	132
typed tool calls	1,106	824 MB	62–127 s	3.4 MB	11–18 μs	206	189	89	87
96 fields	1,970	1.5 GB	not built	3.85 MB	8–14 μs	—	260	—	121

Table 20 | Rows on demand (host-side variant, Qwen3.5-2B, synchronous engine, no healing): rows computed by one decode and their memory against the full table.

					Tokens/s		
Grammar	States	Full table	Rows	Row memory	full	warm	cold
person	104	77 MB	11	8 MB	129	124	95
tool_call	164	122 MB	9	7 MB	224	210	159
product	192	143 MB	21	16 MB	154	158	133
event	178	133 MB	17	13 MB	138	136	106
classification	158	118 MB	13	10 MB	135	139	123
contact_card	246	183 MB	16	12 MB	147	150	114
48 fields	986	735 MB	85	63 MB	215	222	160
typed tool calls	1,106	824 MB	17	13 MB	148	164	94
96 fields	1,970	not built	85	63 MB	—	222	159

Table 21 | Constraint memory and compile time across engines. Qwen3.5 vocabulary, CPU-side measurement, each library in its own process, every grammar driven through a valid document computing the mask at every step. Shared: memory paid once per process whatever the number of grammars.

	STCON table	STCON walk	llguidance	XGrammar
person schema (104 states)	77 MB	0.05 MB	<0.5 MB	0.18 MB
96 fields (1,970 states)	1.5 GB	1.0 MB	<0.5 MB	0.51 MB
typed tool calls (1,106 states)	824 MB	0.57 MB	fails	5.6 MB
recursive schema (solar)	38 MB	—	<0.5 MB	0.17 MB
16-grammar server	7.2 GB, dies at the 15th	4.9 MB	4.7 MB	4.4 MB
shared, once per process	—	**2.9 MB**	153 MB	≈9 MB
compile, typed tool grammar	25 s DFA + 62–127 s table	25 s DFA	—	0.28 s
compile, one schema	0.7–1.7 s	0.02–0.4 s	1–2 ms	1–19 ms
mask, per step	GPU gather	7–18 μs (GPU)	60–270 μs (CPU)	3–13 μs (CPU)

Table 22 | The sixteen-grammar server, both models and two prompt-cache sizes. The grammars are the six schemas, then schemas of 8, 12, 16, 24, 32, 40, 48, 64, 80 and 96 fields; four requests per grammar, three concurrent.

Model	Prompt cache	Constraints	Served	Above the model	Longest first request
2B	50 MB	token tables	15/16, then out of memory	8.2 GB	26 s
		walk kernel	**16/16**	**0.6 GB**	**2.8 s**
2B	1.5 GB	token tables	14/16, then out of memory	7.6 GB	20 s
		walk kernel	**16/16**	**2.2 GB**	**3.1 s**
4B	50 MB	token tables	14/16, then out of memory	6.9 GB	26 s
		walk kernel	**16/16**	**1.1 GB**	**6.6 s**

Table 23 | Peak GPU memory above the model by decoding regime. Qwen3.5-4B, B streams $\times$ N tokens on LiveCodeBench prompts, no constraint.

	STCON		Stock mlx-lm	
Streams $\times$ tokens	above model	tok/s	above model	tok/s
1×4096	1.0 GB	65	0.8 GB	60
8×64 (prefill only)	6.4 GB	—	3.8 GB	—
8×1024	6.4 GB	175	3.7 GB	82
8×4096	6.4 GB	212	5.2 GB	84
6×8192	4.9 GB	93	5.9 GB	64
6×16384	4.9 GB	128	out of memory	—

Table 24 | The prefill transient (Qwen3.5-4B, our engine, nothing decoded): peak memory above the model while prefilling the longest prompt (1,360 tokens), alone and in right-padded batches, by prefill chunk size.

Prefill chunk	2048	1024	512	256	128
one stream, 1,360 tokens	1.41 GB	1.46 GB	1.23 GB	1.09 GB	1.19 GB
batch of 2		2.12 GB	1.36 GB		
batch of 4		3.50 GB	1.59 GB		
batch of 8		6.48 GB	2.43 GB		

At short contexts the peak is the prompt pass, not the caches: our batch engine prefills every admitted prompt in one right-padded batch with 1,024-token chunks, about 0.8 MB per token of (rows $\times$ chunk), while stock prefills four prompts at a time in 512-token steps but computes logits at every prompt position. Capping a prefill pass at about 2,000 tokens brings our 8-slot peak to about 2.4 GB. At long contexts our caches never exceed the prefill transient, while stock grows its KV cache by copying, so its growth costs about twice the raw cache and exceeds the wired limit at six 16k-token streams.

A memory budget for n agents. With W the weights, C the prompt-cache budget, P the prefill transient, n streams of T tokens and G the constraints, the peak is about $M \approx W + C + P + n(s + \tau T) + G$. On the 4B, $W = 2.26$ GB, $s = 50$ MB, $\tau = 32$ KB per token, $P \approx 0.8$ MB $\times$ (rows $\times$ chunk), and G is 3 MB per grammar with the walk kernel against $3|Q||\mathcal{V}|$ bytes, never freed, with tables. Eight coding agents with 1,000-token prompts, 4,000-token answers and four schemas need $2.26 + 1.5 + 6.4 + 8 \times (0.05 + 0.13) + 0.01 \approx 11.6$ GB on our engine today and about 7.6 GB with the 2,000-token prefill cap; on stock mlx-lm with tables for the four schemas, about 10.8 GB, at the edge of the 10.7 GB limit.

I Budgets, answer quality and exactness

I.1 Budget masking

Table 25 | Validity under a token budget, regular path (Qwen3.5-2B). Budgets $B = 2, 4, \ldots$ up to the full greedy answer; need_0 is the budget table's value at the start, the shortest valid output under the forced-span policy. Feasible budgets are those with $B \geq \text{need}_0$.

Schema	Full answer	need_0	First valid	Feasible	Valid, mask	Valid, no mask
person	34	16	16	10	10	1
tool_call	29	25	26	2	2	0
product	57	37	38	10	10	0
event	75	18	18	29	29	0
classification	58	22	22	19	19	1
contact_card	74	35	36	20	20	1
all				90	**90**	3

None of the 72 infeasible budgets yields a valid output with or without the mask, as Theorem D.11 predicts, the first valid budget is need_0 or the next even number. On the pushdown path (four completable recursive tasks, budgets in steps of six from eight tokens) the byte-level bound of Proposition D.13 gives 54 of 61 valid outputs against 2 of 61, every budget valid from the first one that yields a valid output (12–20 tokens) upward.

I.2 Answer quality

Agreement is the share of leaf values of the model's unconstrained greedy answer that the constrained output reproduces exactly, pooled over the six schemas (the per-workload mean is 0.94 / 0.87 for STCON on the 2B / 4B). It measures fidelity to what the model would have said, not correctness, and penalizes harmless variants. *Gold correctness* compares 45 objective fields of five schemas with values read off the source texts: person (8 fields), tool_call (5), product (11), event (12) and contact_card (9). The classification task has no single correct answer and is excluded. Strings are compared after trimming and lowercasing, numbers numerically. Every engine and mode (free, STCON, llguidance in both modes) reproduces all 45 gold fields on both models. Table 26 compares segmentation policies on the same constraint.

Limit	Output (Qwen3.5-2B, event schema)
320 (75 used)	`{"title":"The Applied AI Summit","date":"2026-11-12","location":{"venue":"Congress Center","city":"Berlin","country":"Germany"},"attendees":[{"name":"Dr. Aiko Tanaka","role":"speaker"},{"name":"Lars Petersen","role":"speaker"},{"name":"Nina Weber","role":"organizer"}]}`
48	`{"title":"The Applied AI Summit","date":"2026-11-12","location":{"venue":"Congress Center","city":"Berlin","country":"Germany"},"attendees":[{"name":"Dr.","role":"speaker"}]}`
32	`{"title":"The Applied AI Summit","date":"2026-11-","location":{"venue":"","city":"","country":""},"attendees":[]}`
24	`{"title":"The Applied AI Summit","date":"","location":{"venue":"","city":"","country":""},"attendees":[]}`
40, no rule	`{"title":"The Applied AI Summit","date":"2026-11-12","location":{"venue":"Congress Center","city":"Berlin","country":"Germany"},"attendees":[{"name":"` (cut mid-value: invalid)

Fig. 7 | The budget rule on one output, unabridged. As the limit tightens the decoder closes the array early, then shortens strings, then empties them, because it admits only tokens after which a valid ending still fits.

Table 26 | Feeding forced text in the tokenizer's own split restores the model's answers and needs the fewest runs. Qwen3.5-2B, six schemas, the same constraint throughout; only the way forced bytes become tokens differs. Agreement is the share of leaf values that match the model's unconstrained answer.

How forced text is fed	Engine	Agreement	Model runs
one token per run, mask only	llguidance	0.95	341
forced tokens skipped	llguidance (fast-forward)	0.95	245
model chooses the split	STCON, plain masking	0.93	343
whole chain in any split	STCON, commit-all	0.93	246
tokenizer's own split	STCON, canonical policy	**0.95**	**224**

In the segmentation-policy run only the policy differs, so the "Just-Luc" divergence under plain token-level masking is due to the policy.

J What did not help

Table 27 | Measured ceilings and negative results (Qwen3.5-2B). Every candidate was first bounded by replacing the component with a free stand-in and timing the real step.

Idea	Measurement and decision
Defer the recurrent-state writes (a log of updates)	removing the state store saves 2.3% of a step at batch 1, 6.7% at 16–32; not built
A certified low-rank vocabulary head	the head's spectrum is flat: rank 1,024 of 2,048 rules out under 2% of rows; not built
Fuse the norms, residual adds and SwiGLU	2–10% of a pipelined step; each costs 1–2 μs
Score few-choice regions as a tree in one pass	forced spans already absorb them: 0–4% fewer passes, no wall-time change
Finite regions (sequence-level choice of a label among 16)	2.0 passes against 2.9, but the intended label on 7 of 24 tickets against 15 for token-level greedy: the most probable complete string is the short generic label
Decode sibling subtrees of a recursive array in parallel	slower on every task (Table 28); branches invent what their siblings decide
Speculative drafts in the batch engine	a 2-token step at 8 rows costs $1.48\times$ a 1-token step: batched steps are compute-bound
Prompt-lookup drafts verified against the constraint	exact; $1.02\times$ overall, 1.1–$1.27\times$ where the model copies, up to 11% slower where drafts are rejected; off by default
Block-parallel fields without a certificate	$1.76\times$ end to end on 18 requests at agreement 0.94, but approximate: a value does not see earlier values

The block product holds for the language of sibling subtrees; it does not make the model's decisions independent. That is why the certified block decoder verifies its drafts, and why sibling-parallel decoding without verification fails.

Table 28 | **Sibling-parallel decoding of recursive arrays fails without verification** (Qwen3.5-2B; wall time is prefill plus decode).

Task	Sequential pushdown	Sibling-parallel
solar	0.84 s, 109 tok/s, 11 nodes	2.02 s, 66 tok/s, 13 nodes (Venus given Earth's subtree)
thread	2.67 s, 143 tok/s	2.67 s, 71 tok/s (a reply duplicated)
fs	0.85 s, 122 tok/s	4.03 s, 82 tok/s
cats	0.55 s, 76 tok/s	2.00 s, 55 tok/s
org	0.32 s, 79 tok/s	2.09 s, 89 tok/s
menu	0.39 s, 104 tok/s	35 s, 1,168 invented nodes